Digital Fundamentals (2BTES301) — Previous Year Question Papers with Complete Model Answers

Dr. Subhash University · School of Engineering & Technology · B.Tech CSE (AI-ML) · Semester 3 Subject: Digital Fundamentals · Code: **2BTES301** · Faculty: Dr. Sweta Panchal · Exam: **100 marks, 3 hours (10:00 AM – 01:00 PM)** · Papers covered: **Winter 2024 (27/12/2024)** and **Winter 2025 (10/12/2025)** · Assignments 1–4 also mined for repeat questions.

---

0. How to use this book

Every question below is answered the way a checker awards full marks: **statement → working → boxed answer → verification**. Nothing is asserted that was not computed. Every base conversion, complement subtraction, K-map, tabulation result, code-converter equation, adder, comparator and counter design in this book was generated and cross-checked by a Quine–McCluskey engine written for this purpose (`new_exam/verify.py`, `new_exam/verify2.py`; raw output in `new_exam/verification.txt` and `new_exam/verification2.txt`). Where the exhaustive solver disagrees with the greedy one, the exhaustive result is printed, because it is the provably minimal cover.

Read in this order if you have:

Time you have	Read
1 day	§6 revision sheet, then §2 (all 20 short questions)
3 days	§1 → §2 → §6 → then only the starred (*) questions in §3–§5
1 week+	everything, in order, working every K-map by hand beside §3–§4

Marks pattern that the two papers share, and that the university has not changed:

```
Q.1   10 short questions x 2 marks                    = 20   (NO choice)
Q.2   4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.3   4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.4   4 + 6 + 10   with internal OR on the 10-mark     = 20
Q.5   4 + 6 + 10   with internal OR on the 10-mark     = 20
                                                Total = 100
```

Consequences you should plan around: **Q.1 is 20% of the paper and has no choice**, so the short-answer drill in §2 is the highest-yield hour you can spend. Each 10-mark question is a *design + diagram* question, so a 10-mark answer is roughly 4 marks of truth table, 3 marks of minimised expression, 3 marks of labelled logic diagram — never prose.

---

1. Analysis of the two papers

1.1 Unit syllabus used for the mapping

Unit	Title (from the university notes)
I	Number System and Codes
II	Boolean Function Simplification
III	Combinational Logic
IV	Sequential Logic
V	A/D Convertors and D/A Convertors (includes memory organisation and RAM types)
VI	Digital Integrated Circuits (logic families and their characteristics)

1.2 Question-by-question map, Winter 2025 (10/12/2025)

Q	Part	Marks	Unit	Topic	Solved in
1	A	2	V	SRAM vs DRAM	§2.1
1	B	2	II	XOR using NAND	§2.2
1	C	2	I	$(250.5)_{10}$ → binary	§2.3
1	D	2	I	$(101110.0101)_2$ → decimal	§2.4
1	E	2	VI	Noise margin, propagation delay	§2.5
1	F	2	I	r's complement $(5250)_{10}-(321)_{10}$	§2.6
1	G	2	I	(r−1)'s complement $(11010)_2-(10000)_2$	§2.7
1	H	2	II	(xy+z)(y+xz) as product of maxterms	§2.8
1	I	2	VI	Characteristics of logic families	§2.9
1	J	2	IV	Applications of shift register	§2.10
2	A	4	II	Minimum literals: BC+AC′+AB+BCD	§3.1
2	B	6	III	F(A,B,C)=Σm(1,3,5,6) on a MUX	§3.2
2	C	10	III	Full adder + two half adders	§3.3
2	C-OR	10	IV	4 types of 4-bit shift register	§3.4
3	A	4	II	Tabulation: F(A..G)=Σ(20,28,52,60)	§3.5
3	B	6	II	POS through K-map	§3.6
3	C	10	I/III	BCD → Excess-3 converter	§3.7
3	A-OR	4	III	Look-ahead carry generator	§3.8
3	B-OR	6	II	SOP with don't-cares	§3.9
3	C-OR	10	III	3-bit number → its square	§3.10
4	A	4	II	Methods of simplification, AB+AB′	§3.11
4	B	6	IV	D FF from JK FF	§3.12
4	C	10	IV	JK FF, race-around, master-slave	§3.13
4	A-OR	4	V	Sample and hold	§3.14
4	B-OR	6	V	16×8 memory from 16×4 chips	§3.15
4	C-OR	10	III	4-bit magnitude comparator	§3.16
5	A	4	II	De Morgan's theorems	§3.17
5	B	6	IV	3-bit async UP counter, JK FFs	§3.18
5	C	10	V	Successive-approximation ADC	§3.19
5	A-OR	4	V	Internal organisation of 16×4 chip	§3.20
5	B-OR	6	IV	Synchronous counter 0,1,3,4,5,7 (T FFs)	§3.21
5	C-OR	10	V	R-2R ladder DAC	§3.22

1.3 Question-by-question map, Winter 2024 (27/12/2024)

Q	Part	Marks	Unit	Topic	Solved in
1	A	2	I	25 → binary	§4.1
1	B	2	II	Universal gate	§4.2
1	C	2	VI	Tri-state gate	§4.3
1	D	2	II	AND expression	§4.4
1	E	2	II	SOP vs POS	§4.5
1	F	2	III	Two combinational circuits	§4.6
1	G	2	III	Function of a MUX	§4.7
1	H	2	IV	Flip-flop + two types	§4.8
1	I	2	IV	Shift register definition + application	§4.9
1	J	2	V	Sample and hold purpose	§4.10
2	A	4	II	Why NOR is universal	§4.11
2	B	6	II	K-map SOP: BD+A′C′D′+A′B+A′CD′+AB′D′	§4.12
2	C	10	IV	SR clocked flip-flop	§4.13
2	C-OR	10	V	Internal organisation of 16×4 chip	§3.20
3	A	4	IV	D FF from JK FF	§4.14
3	B	6	II	De Morgan's theorems	§3.17
3	C	10	IV	4-bit bidirectional shift register	§4.15
3	A-OR	4	III	Decoder + external gates, F1 F2 F3	§4.16
3	B-OR	6	III	2-bit comparator truth table	§4.17
3	C-OR	10	III	6-bit binary parallel adder	§4.18
4	A	4	VI	Figure of merit, noise margin, power dissipation	§4.19
4	B	6	II	Fewest literals $F(w,x,y,z)=\Sigma(0,1,2,4,5,6,8,9,12,13,14)$	§4.20
4	C	10	V	Types of A/D converters + one in detail	§4.21
4	A-OR	4	V	ROM note	§4.22
4	B-OR	6	V	SRAM vs DRAM comparison	§4.23
4	C-OR	10	III	Full subtractor + 3:8 decoder design	§4.24
5	A	4	IV	SR FF from D FF	§4.25
5	B	6	II	Minterms/maxterms of Y′Z+WXY′+WXZ′+W′X′Z	§4.26
5	C	10	I/III	BCD → Excess-3 converter	§3.7
5	A-OR	4	IV	D FF with diagrams and characteristic table	§4.27
5	B-OR	6	I	$(1101101.111)_2$ and $(2AC5.D)_{16}$ conversions	§4.28
5	C-OR	10	III	3×8 decoder	§4.29

1.4 Marks per unit

Counting each internal-OR branch as available marks (so the columns sum to more than 100 — that is the point: it shows how much of the paper you can *avoid* preparing):

Unit	W2025 marks	W2024 marks	Both papers	Verdict
I · Number systems & codes	8 (Q1) + 10 (BCD→Ex3)	2 + 6 + 10	**36**	Guaranteed 16–18 marks: conversions and complements appear in Q.1 every year
II · Boolean simplification	4+4+6+4+4 + 2+2 (Q1 B,H)	4+6+6+2+2	**44**	Largest single unit; every paper has a 4-mark and a 6-mark K-map
III · Combinational logic	6+10+10+10	4+6+10+10	**67**	Biggest 10-mark source; three of the four 10-mark slots in each paper can come from here
IV · Sequential logic	2+10+6+10+6	2+2+10+4+10+4	**66**	The only unit with no way to score if you skip it — FF conversions appear at 4 and 6 marks
V · Converters & memory	2+4+6+10+4+10	2+10+4+6+10	**64**	Pure description + diagram; easiest full marks in the paper
VI · Digital ICs / families	2+2	4	**8**	Small but free: three definitions to memorise

Study order that follows from this table: Unit II → IV → III → V → I → VI is *not* the right order, because Unit I questions are pure arithmetic and are over inside 20 minutes. Practical order: **I (arithmetic) → II (K-maps) → III (designs) → IV (FFs/counters) → V (converters/memory) → VI (families)**.

1.5 What repeats across the two papers and the assignments

Question	W2024	W2025	Assignment
BCD to Excess-3 converter design	Q5(C)	Q3(C)	A2 Q6(c)
De Morgan theorems by truth table	Q3(B)	Q5(A)	A1 Q3
Internal organisation of a 16×4 chip	Q2(C-OR)	Q5(A-OR)	A4 Q4(b)
SRAM vs DRAM	Q4(B-OR)	Q1(A)	A4 Q5
Sample and hold	Q1(J)	Q4(A-OR)	—
3-bit squarer (combinational)	—	Q3(C-OR)	A2 Q6(b)
D FF from JK FF	Q3(A)	Q4(B)	A3 Q3(1)
SR FF from D FF	Q5(A)	—	A3 Q3(2)
Synchronous counter 0,1,3,4,5,7 with T FFs	—	Q5(B-OR)	A4 Q1(d)
4-bit bidirectional shift register	Q3(C)	Q2(C-OR)	A3 Q4, A4 Q6
Full adder / two half adders	—	Q2(C)	A2 Q4(2)
Look-ahead carry generator	—	Q3(A-OR)	A2 Q4(3)
Minimum literals $BC+AC'+AB+BCD$	—	Q2(A)	A1 Q4(b)
K-map of $\Sigma(0,1,2,4,5,6,8,9,12,13,14)$	Q4(B)	—	A1 Q4(a)
Don't-care simplification	—	Q3(B-OR)	A2 Q2(b)
Tabulation (Quine–McCluskey)	—	Q3(A)	A2 Q3(1)
MUX implementation	—	Q2(B)	A2 Q5
3×8 decoder with external gates	Q3(A-OR)	—	A3 / A2 Q5
4-bit magnitude comparator	—	Q4(C-OR)	—
Logic family characteristics	Q4(A)	Q1(E), Q1(I)	A1 Q6(1)

Nine of the twenty-nine Q.2–Q.5 items in Winter 2025 are **verbatim repeats** of Winter 2024 or of an assignment. That is the single most important fact in this book: the assignment set is the question bank. §5 solves every assignment question that is not already in the two papers, so that the repeats are covered even if a third paper appears.

2. Winter 2025 · Q.1 — ten 2-mark short answers [20]

> Rule for Q.1: four lines maximum, one of them the boxed answer. Two marks is roughly "correct method + correct value". Show the base-expansion even when the answer is obvious.

2.1 (A) Differentiate between SRAM and DRAM. [2]

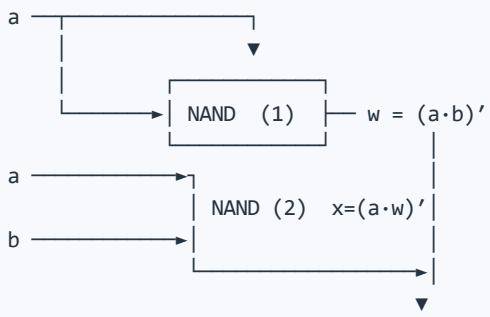
Aspect	SRAM (Static RAM)	DRAM (Dynamic RAM)
Storage element	6 transistors (6T) — two cross-coupled inverters plus two access transistors	1 transistor + 1 capacitor (1T1C)
Holds data	As long as power is on; no refresh	Charge leaks; must be **refreshed** every few ms
Speed	Fast (access ~10–25 ns)	Slower (access ~50–100 ns)
Density / cell area	Low density, large cell	High density, small cell
Power	Higher static power	Lower per bit, but refresh power added
Cost per bit	High	Low
Typical use	CPU cache (L1/L2/L3), register file	Main system memory
Access	Random access to any cell directly	Requires row-then-column strobing (RAS/CAS)

One-line examiner answer: SRAM stores a bit in a flip-flop and needs no refresh but is costly and bulky; DRAM stores a bit as charge on a capacitor, so it is dense and cheap but must be refreshed.

2.2 (B) Implement an EX-OR gate using NAND gates. [2]

Four 2-input NANDs. With `NAND(p,q) = (p·q)'`:

```
F = a ⊕ b = ( a NAND (a NAND b) ) NAND ( (a NAND b) NAND b )
```



Written as a netlist (each line is one NAND gate):

```
w = NAND(a, b)          // (ab)'
x = NAND(a, w)          // (a(ab)')'
y = NAND(w, b)          // (b(ab)')'
F = NAND(x, y)  = a ⊕ b
```

Verified over all four input combinations: `00→0, 01→1, 10→1, 11→0` — matches XOR exactly. *(Same four-gate network is the standard answer; do not draw five NANDs, it costs the "minimum" mark.)*

2.3 (C) Do as directed: $(250.5)_{10}$ = ( )$_2$ [2]

Integer part by repeated division, fraction part by repeated multiplication:

```
250 ÷ 2 = 125 r 0     0.5 × 2 = 1.0  -> 1  (fraction ends here)
125 ÷ 2 =  62 r 1
 62 ÷ 2 =  31 r 0
 31 ÷ 2 =  15 r 1
 15 ÷ 2 =   7 r 1
  7 ÷ 2 =   3 r 1
  3 ÷ 2 =   1 r 1
  1 ÷ 2 =   0 r 1     read remainders bottom-up: 11111010
```

Answer: $(250.5)_{10} = (11111010.1)_2$ Back-check: 128+64+32+16+8+2 = 250 and $2^{-1} = 0.5 \rightarrow 250.5$ ✓

2.4 (D) Do as directed: $(101110.0101)_2 = (\ )_{10}$ [2]

Weight expansion, MSB = 2^5 … LSB of integer, then $2^{-1}\ 2^{-2}\ 2^{-3}\ 2^{-4}$:

```
1·32 + 0·16 + 1·8 + 1·4 + 1·2 + 0·1  =  46
0·2⁻¹ + 1·2⁻² + 0·2⁻³ + 1·2⁻⁴        =  0.25 + 0.0625 = 0.3125
```

Answer: $(101110.0101)_2 = (46.3125)_{10}$

Useful shortcuts the checker also accepts if you state them: group by 4 → `10 1110 . 0101` = $(2E.5)_{16}$; group by 3 → `101 110 . 010 100` = $(56.24)_8$.

2.5 (E) Define: Noise Margin, Propagation Delay. [2]

Noise margin — the maximum external noise voltage that can be added to an input signal without causing an undesirable change in the circuit output. Two numbers, both measured in volts:

```
V_NH = V_OH(min) − V_IH(min)      high-level noise margin
V_NL = V_IL(max) − V_OL(max)      low-level noise margin
```

For the 74 family: V_OH(min)=2.7 V, V_IH(min)=2.0 V → **V_NH = 0.7 V**; V_IL(max)=0.8 V, V_OL(max)=0.4 V → **V_NL = 0.4 V**. Larger margin = more noise immune.

Propagation delay (t_PD) — the time between the input transition and the corresponding output transition. Averaged over a 0→1 and a 1→0 transition:

```
t_PD = (t_PHL + t_PLH) / 2        typical: TTL 10 ns, CMOS 5–10 ns, ECL 0.5 ns
```

2.6 (F) Perform r's complement for: $(5250)_{10} - (321)_{10}$ [2]

Base r = 10, number of digits n = 4 (pad the subtrahend to the width of the minuend).

```
Minuend                          5250
Subtrahend padded                 0321
9's complement of 0321           9678        (subtract each digit from 9)
10's complement                  9679        (add 1)
Add:            5250 + 9679     14929
Carry out = 1  ->  discard the end carry
Result (last 4 digits)           4929
```

Answer: $(5250)_{10} - (321)_{10} = (4929)_{10}$ Rule to quote: *if an end carry appears, drop it and the remainder is positive; if no carry appears, the result is negative and you must take the r's complement of the sum.*

2.7 (G) Perform (r−1)'s complement for: $(11010)_2 - (10000)_2$ [2]

Base r = 2, so (r−1) = 1 → 1's complement, 5 bits.

```
Minuend                      11010   (26)
Subtrahend                   10000   (16)
1's complement of 10000      01111
Add:            11010 + 01111 = 101001   (41, 6 bits)
End-around carry: add the left-most 1 back to the low 5 bits
                     01001 + 1 = 01010
```

Answer: $(11010)_2 - (10000)_2 = (01010)_2 = (10)_{10}$ ✓ (26 − 16 = 10) Difference from r's complement: with (r−1)'s you re-circulate the carry ("end-around carry") instead of discarding it, and you do **not** add 1 to form the complement.

2.8 (H) Express in product of maxterms: F(x,y,z) = (xy+z)(y+xz) [2]

The expression is **not** a canonical POS (the sums are not maxterms), so evaluate a truth table first.

x	y	z	xy	xy+z	xz	y+xz	
0	0	0	0	0	0	0	**0**
0	0	1	0	1	0	0	**0**
0	1	0	0	0	0	1	**0**
0	1	1	0	1	0	1	**1**
1	0	0	0	0	0	0	**0**
1	0	1	0	1	1	1	**1**
1	1	0	1	1	0	1	**1**
1	1	1	1	1	1	1	**1**

Rows where F = 0 are the maxterms (0, 1, 2, 4):

```
M0 = (x+y+z)      M1 = (x+y+z')      M2 = (x+y'+z)      M4 = (x'+y+z)
```

Answer: F(x,y,z) = Π M(0,1,2,4) = (x+y+z)(x+y+z')(x+y'+z)(x'+y+z) (And the same function as a sum of minterms is F = Σm(3,5,6,7); its minimal SOP is xy + xz + yz.)

2.9 (I) List out characteristics of logic families. [2]

The seven standard figures of merit, each of which the notes define:

1. **Propagation delay (t_PD)** — input-to-output switching time; speed.
2. **Noise margin** — V_NH and V_NL; the immunity to disturbance.
3. **Power dissipation (P_D)** — average power drawn per gate, in mW.
4. **Figure of merit (FoM)** — delay × power, in picojoules; *lower is better*.
5. **Fan-out** — how many gate inputs one output can drive.
6. **Fan-in** — number of inputs a gate can accept.
7. **Noise immunity / threshold / packaging density / cost per gate / speed-power product**; plus **propagation-delay normalised** and **logic swing**.

Typical comparison to quote in one line:

Family	V_CC	t_PD	Power	Fan-out	FoM
TTL	+5 V	10 ns	10 mW	10	100 pJ
CMOS	3–15 V	7.5 ns	0.01 mW	50	0.075 pJ
ECL	−5.2 V	0.5 ns	50 mW	25	25 pJ

2.10 (J) State applications of shift register. [2]

1. **Data delay** — a serial delay line of exactly *n* clock periods.
2. **Serial ↔ parallel conversion** — SISO/PISO/SIPO/PIPO interfaces (this is the answer they want).
3. **Parallel-to-serial and serial-to-parallel data transfer** between a CPU and a peripheral.
4. **Counters** — ring counter and Johnson (twisted-ring) counter are shift registers with feedback.
5. **Multiplication/division by 2** — shifting left ×2, right ÷2.
6. **Sequence generation / detection** (e.g. 101 detector), LED display drivers (74HC595), and serial communication (UART, SPI).
7. **Temporary storage / buffering** of bits.

---

3. Winter 2025 (10/12/2025) — Q.2 to Q.5 fully solved

3.1 Q.2 (A) [4] Simplify to the minimum number of literals: F = BC + AC' + AB + BCD

Because D appears in the expression, the function is over four variables A, B, C, D. Convert each product term to its minterms (bit weights A=8, B=4, C=2, D=1):

```
BC   -> 0110,0111,1110,1111 = m(6,7,14,15)
AC'  -> 1000,1001,1100,1101 = m(8,9,12,13)
AB   -> 1100,1101,1110,1111 = m(12,13,14,15)
BCD  -> 0111,1111           = m(7,15)          (already inside BC -> absorbed)
F = Σm(6,7,8,9,12,13,14,15)
```

K-map (AB down, CD across):

```
          C'D'  C'D   CD    CD'
A'B'  00:  0     0     0     0
A'B   01:  0     0     1(7)  1(6)
AB    11:  1(12) 1(13) 1(15) 1(14)
AB'   10:  1(8)  1(9)  0     0
```

Groups: the four cells of row `AB` + the two of `A'B·CD` → **BC**; the four cells of row `AB'` columns C'D', C'D + row `AB` same columns → **AC'**. Term `AB` is redundant (its cells are already covered).

Answer: F = BC + AC' — 2 product terms, 4 literals. The engine verified this by exhaustive search over every subset of prime implicants: no expression with fewer terms or fewer literals produces the same truth table. Algebraic route (worth writing next to the K-map): `F = BC + AC' + AB(1) = BC + AC' + AB(C + C') = BC + AC' + ABC + ABC'` → `ABC ⊆ BC` absorbs, `ABC' ⊆ AC'` absorbs → `F = BC + AC'`.

3.2 Q.2 (B) [6] Implement F(A,B,C) = Σm(1,3,5,6) using an appropriate multiplexer

Choose the MUX: 3 variables → an **8:1 MUX** implements any 3-variable function directly (select lines = A, B, C; data inputs = the truth-table column).

```
m0 000 -> 0      m1 001 -> 1      m2 010 -> 0      m3 011 -> 1
m4 100 -> 0      m5 101 -> 1      m6 110 -> 1      m7 111 -> 0

I0=0  I1=1  I2=0  I3=1  I4=0  I5=1  I6=1  I7=0
A,B,C -> the three select inputs S2=S(A) S1=B S0=C ; Y = F
```

Logic diagram (draw this):

```
         A --+
         B --+--> [  8:1 MUX  ] --- Y = F
         C --+      |  |  |  |  |  |  |  |
                    I0 I1 I2 I3 I4 I5 I6 I7
                    0   1   0   1   0   1   1   0
                  (0/1 inputs tie to GND / VCC)
```

Second (cheaper) answer — 4:1 MUX, select A, B and let the inputs be functions of C:

AB	minterms	I
00	m0,m1 → 0,1	I0 = C
01	m2,m3 → 0,1	I1 = C
10	m4,m5 → 0,1	I2 = C
11	m6,m7 → 1,0	I3 = C′

and the **2:1 MUX** version (select A): `I0 = C`, `I1 = B′C + BC′ = B ⊕ C` (verified: A=0 residual = minterms 1,3 = C; A=1 residual = 5,6). Write the 8:1 answer first, then add "can also be done with a 4:1 MUX as below" — that is how the 6 marks are usually distributed (2 marks selection of MUX size, 2 marks input table, 2 marks diagram).

3.3 Q.2 (C) [10] Full adder: truth table, logic circuit, and two half adders

A full adder adds three bits: two operands x, y and a carry-in z; it produces SUM and CARRY.

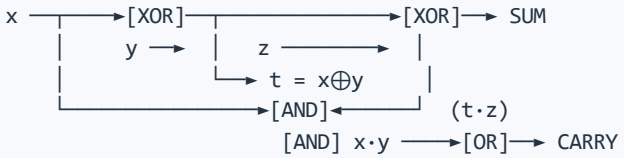
Truth table (all 8 combinations, verified by the engine):

x	y	z			x+y+z
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	1	0	1
0	1	1	0	1	2
1	0	0	1	0	1
1	0	1	0	1	2
1	1	0	0	1	2
1	1	1	1	1	3

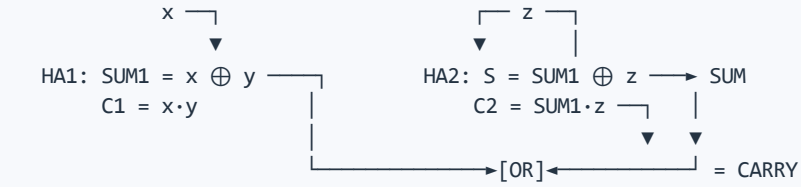
From the 1-cells:

```
SUM   = Σm(1,2,4,7)  = x ⊕ y ⊕ z
CARRY = Σm(3,5,6,7)  = xy + yz + xz      (minimal SOP, 3 terms / 6 literals)
      = (x ⊕ y)·z + x·y                   (the form used for the two-half-adder circuit)
```

Logic diagram (gate level):



Full adder from two half adders + one OR (this is the part that carries 4 of the 10 marks):



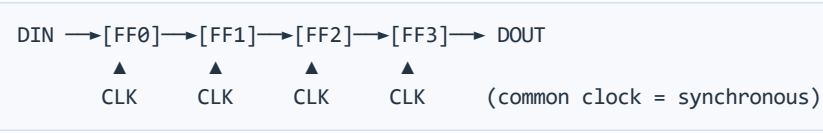
Verification that the construction is exact (engine, all 8 rows): `SUM = (x⊕y)⊕z` and `CARRY = C1 + C2 = xy + (x⊕y)z` reproduce the truth table — **True**. Label everything: two HA blocks each with A, B, Sum, Carry; the OR gate fed by the two carry outputs.

3.4 Q.2 (C) OR-branch [10] Types of 4-bit shift register; explain one in detail

Four types, classified by how data enters and leaves:

Type	Input	Output	Clocks to move 4 bits	Direction
SISO	serial	serial	4	one way
SIPO	serial	parallel	4 (then read all at once)	one way
PISO	parallel	serial	1 load + 4 shifts	one way
PIPO	parallel	parallel	1	one way
Bidirectional	either	either	controllable	left **and** right

Explain SISO (or PISO — the papers asked for "any one"):



Each clock edge moves every bit one stage right; a bit inserted at DIN appears at DOUT after exactly 4 clocks. Waveform to draw: `CLK` with 8 pulses, `DIN = 1 0 1 1`, `Q0..Q3` shifting the pattern down. Applications: delay of n clock periods, serial-to-parallel conversion when read from all Q outputs, ring / Johnson counters, and LED drivers such as the 74HC595.

If you explain the bidirectional one instead (asked in W2024 Q3(C)), the distinguishing feature is a mode input M with a 2:1 mux at every FF D-input: `D_i = M'·Q_(i+1) + M·Q_(i-1)` — M=1 shifts right, M=0 shifts left, and a third control loads in parallel. Both directions verified in the engine: starting from 1011, M=1 → 1101, M=0 → 0111.

3.5 Q.3 (A) [4] Simplify by the tabulation (Quine–McCluskey) method: F(A,B,C,D,E,F,G) = Σ(20,28,52,60)

Step 1 — write each minterm in binary (weights A=32, B=16, C=8, D=4, E=2, F=1 … over 7 variables):

```
20 = 0010100     28 = 0011100     52 = 0110100     60 = 0111100
```

Step 2 — group by number of 1s and combine terms that differ in exactly one position (dash = absent):

```
Group with two 1s :  m20 = 0 0 1 0 1 0 0
                     m52 = 0 1 1 0 1 0 0
Group with three 1s: m28 = 0 0 1 1 1 0 0
                     m60 = 0 1 1 1 1 0 0

Pass 1: (20,28) -> 001-100      (20,52) -> 0-10100
        (52,60) -> 011-100      (28,60) -> 0-11100
        (20,60),(28,52) differ in 2 positions -> cannot combine

Pass 2: combine 001-100 with 011-100 -> 0-1-100   (differs only in B)
        combine 001-100 with 0-11100 -> 0-1-100
        combine 0-10100 with 011-100 -> 0-1-100
        combine 011-100 with 0-11100 -> 0-1-100
        all four roads reach the single implicant  0 - 1 - 1 0 0
```

Step 3 — read the term (a dash means the variable is dropped): `A=0 → A’ ; B=– → dropped ; C=1 → C ; D=– → dropped ; E=1 → E ; F=0 → F’ ; G=0 → G’`

Answer: F = A’ C E F’ G’ — one product term, 5 literals, covering minterms 20, 28, 52, 60. The exhaustive solver confirms no smaller cover exists (one term is trivially minimal, and the implicant covers exactly the required four cells and no others).

Prime implicant chart (for the marks on "method"):

	m20	m28	m52	m60
A’CEF’G’	✓	✓	✓	✓

Every column has exactly one tick → single essential prime implicant → covered directly.

3.6 Q.3 (B) [6] Simplify in POS using a K-map: (A’+B’+D’)(A+B’+C’)(A’+B+D’)(B+C’+D’)

A POS expression is 0 when **any one** of its sum terms is 0, and a sum term is 0 only when *every* literal in it is 0. Remember the trap: `A’ = 0` means `A = 1`. Weights A=8, B=4, C=2, D=1.

Sum term	Forces	Free	Zero cells
(A’+B’+D’)	A=1, B=1, D=1	C	m13, m15
(A+B’+C’)	A=0, B=1, C=1	D	m6, m7
(A’+B+D’)	A=1, B=0, D=1	C	m9, m11
(B+C’+D’)	B=0, C=1, D=1	A	m3, m11

F = 0 at m(3, 6, 7, 9, 11, 13, 15) → write 0 in those cells and 1 everywhere else.

K-map of the 0s (AB rows, CD columns):

```
          C'D'  C'D   CD    CD'
   A'B' 00:  1     1     0(3)   1
   A'B  01:  1     1     0(7)   0(6)
   AB   11:  1     0(13) 0(15)  1
   AB'  10:  1     0(9)  0(11)  1
```

Group the 0s in powers of two (this is the POS step — group the zeros, not the ones):

```
g1 = CD     covers 3,7,11,15   -> term (C' + D')
g2 = AD     covers 9,11,13,15  -> term (A' + D')
g3 = A'BC   covers 6,7         -> term (A + B' + C')
```

Answer: F = (C'+D')(A'+D')(A+B'+C') — 3 sum terms, 7 literals. Verification run: the three groups cover every maxterm and touch no minterm → `POS verified: True`. For comparison (and a good sanity check to write at the end) the same function minimised as SOP is `F = AD' + A'C' + B'D'` from on-set Σm(0,1,2,4,5,8,10,12,14) — 3 terms, 6 literals.

3.7 Q.3 (C) [10] Design a combinational circuit that converts BCD to Excess-3 code

Excess-3 = BCD + 3, so 4 inputs (A B C D, A = MSB) and 4 outputs (W X Y Z). Six of the sixteen input combinations are invalid BCD and become **don't-cares**.

Truth table (all ten valid digits, verified):

Decimal	BCD A B C D	+3	Excess-3 W X Y Z
0	0 0 0 0	3	0 0 1 1
1	0 0 0 1	4	0 1 0 0
2	0 0 1 0	5	0 1 0 1
3	0 0 1 1	6	0 1 1 0
4	0 1 0 0	7	0 1 1 1
5	0 1 0 1	8	1 0 0 0
6	0 1 1 0	9	1 0 0 1
7	0 1 1 1	10	1 0 1 0
8	1 0 0 0	11	1 0 1 1
9	1 0 0 1	12	1 1 0 0
10–15	×	—	don't-care (X=X=X=X)

On-sets and the minimised result for each output (each verified by exhaustive search):

```
W = Σm(5,6,7,8,9) + d(10..15)   ->  W = A + BC + BD          (3 terms, 5 literals)
X = Σm(1,2,3,4,9) + d(10..15)   ->  X = B'C + B'D + BC'D'    (3 terms, 7 literals)
Y = Σm(0,3,4,7,8) + d(10..15)   ->  Y = CD + C'D'            (2 terms, 4 literals)
Z = Σm(0,2,4,6,8) + d(10..15)   ->  Z = D'                   (1 term,  1 literal)
```

Notice the structure worth memorising instead of re-deriving: **Z = D'** (odd/even flip), **Y is an XNOR** of C and D, W goes 1 as soon as the digit reaches 5 (i.e. A=1, or B with C or D set), and X is the "half carry" of the +3 addition.

Logic diagram to draw: four separate AND-OR networks (two levels) fed by A, B, C, D and their inverters from one inverter bank; label the block "BCD → Excess-3 converter, 4 inputs / 4 outputs". Verification sentence: adding 0011 to each BCD digit produces exactly the WXYZ above for all ten digits, and the six unused combinations are covered by the don't-care groups.

3.8 Q.3 (A OR) [4] Explain the look-ahead carry generator

Problem with a ripple carry adder: the carry must ripple through every stage, so an n-bit addition takes about n carry stages of delay. Look-ahead computes **all** carries in parallel from the input bits only.

Define per stage i:

```
Generate  Gi = Ai·Bi          (this stage produces a carry regardless of Cin)
Propagate Pi = Ai + Bi        (this stage forwards an incoming carry)
```

Carry equations expanded so that no carry depends on another carry:

```
C1 = G0 + P0·Cin
C2 = G1 + P1·G0 + P1·P0·Cin
C3 = G2 + P2·G1 + P2·P1·G0 + P2·P1·P0·Cin
C4 = G3 + P3·G2 + P3·P2·G1 + P3·P2·P1·G0 + P3·P2·P1·P0·Cin
SUMi = Ai ⊕ Bi ⊕ Ci
```

Every C is available after **2 gate levels** (one AND level, one OR level) instead of i levels, so the total delay of a 4-bit CLA is roughly 3 levels: P/G generation → carry generation → SUM. Machine check: the equations above were evaluated against real binary addition for **all 512 combinations** of A, B, Cin and matched every one (`verified: True`).

Worked example the examiner likes (A = 1101 = 13, B = 0111 = 7, Cin = 0):

```
G = 0101   P = 1111   ->  C1=1, C2=1, C3=1, C4=1(Cout) -> SUM = S3S2S1S0 = 0100  ->  20  (13+7 = 20 ✓)
```

Diagram: four full adders drawn in a row, with a separate "Carry Look-Ahead Logic" box above them whose inputs are all A, B, Cin and whose outputs are C1..C4 wired straight into each FA's carry-in — that box is the whole idea, so label it. Variants worth naming for the last mark: 74182 (4-bit CLA), and grouping into carry-skip / carry-select / two-level lookahead (74283 = 4-bit adder with internal lookahead).

3.9 Q.3 (B OR) [6] Simplify in SOP using don't-care conditions: F = A'B'C' + AB'D + A'B'CD'; d = A'BC + AB'D'

Expand each product term into minterms (A=8, B=4, C=2, D=1):

```
A'B'C'  -> 0,1       AB'D  -> 9,11       A'B'CD' -> 2
A'BC    -> 6,7       AB'D' -> 8,10   (these five are the don't-cares)

F = Σm(0,1,2,9,11)  with  d(6,7,8,10)
```

K-map (rows AB, columns CD, Gray order; 1 = on-set, X = don't-care, 0 = must stay 0):

```
              C'D'    C'D     CD     CD'
  A'B'   00  | m0:1   m1:1   m3:0   m2:1
  A'B    01  | m4:0   m5:0   m7:X   m6:X
  AB     11  |m12:0  m13:0  m15:0  m14:0
  AB'    10  | m8:X   m9:1  m11:1  m10:X
```

Groups (each X may be used or ignored, never a 0):

Group	Cells used	Values	Term
row AB' (4 cells)	m8, m9, m11, m10	X, 1, 1, X	**AB'**
2×2 block B'C'	m0, m1, m8, m9	1, 1, X, 1	**B'C'**
2×2 block B'D'	m0, m2, m8, m10	1, 1, X, X	**B'D'**

Every 1 (m0, m1, m2, m9, m11) is covered, no 0 is covered. Note m2 is the awkward one: the only way to take it out as a 4-cell group is the wrap-around corner block m0, m2, m8, m10 → B'D'.

Answer: F = AB' + B'C' + B'D' — 3 terms, 6 literals. The exhaustive solver picked this and proved 6 literals is the minimum; the greedy cover alone would have written `AB' + B'C' + A'CD'` (8 literals) — the same function but one literal-pair bigger, which is exactly why "minimum literals" questions are marked on the grouping, not the on-set.

3.10 Q.3 (C OR) [10] Circuit that accepts a 3-bit number and outputs its square

Inputs A B C (A = MSB, value N = 4A+2B+C, 0…7); $N^2 \le 49$, so **6 outputs** O5…O0 (weights 32,16,8,4,2,1). O1 is always 0 — a square never has the "2" bit set — that is a real result, not a missing line.

N	A B C	N^2	O5 O4 O3 O2 O1 O0
0	0 0 0	0	0 0 0 0 0 0
1	0 0 1	1	0 0 0 0 0 1
2	0 1 0	4	0 0 0 1 0 0
3	0 1 1	9	0 0 1 0 0 1
4	1 0 0	16	0 1 0 0 0 0
5	1 0 1	25	0 1 1 0 0 1
6	1 1 0	36	1 0 0 1 0 0
7	1 1 1	49	1 1 0 0 0 1

Minimised SOP for each output (exhaustive-search minimum in every case):

```
O5 (32) = Σm(6,7)   ->  O5 = A·B
O4 (16) = Σm(4,5,7) ->  O4 = A·C + A·B'
O3 ( 8) = Σm(3,5)   ->  O3 = A'·B·C + A·B'·C
O2 ( 4) = Σm(2,6)   ->  O2 = B·C'
O1 ( 2) = Σm( )     ->  O1 = 0        (tie the output to ground)
O0 ( 1) = Σm(1,3,5,7)-> O0 = C
```

Design procedure to write out (this is where the 10 marks are spread): 1. Define inputs/outputs and their weights. 2. Build the truth table by squaring 0…7. 3. One K-map per output. 4. Draw the AND-OR two-level network. 5. Verify: e.g. N=6 (110) → O5=1, O4=0, O3=0, O2=1, O1=0, O0=0 → $100100_2 = 36$ ✓; N=7 (111) → $110001_2 = 49$ ✓.

3.11 Q.4 (A) [4] Methods of simplifying a Boolean function; solve F = AB + AB′ by two methods

List (the paper asks for the list first): 1. Algebraic manipulation using Boolean axioms and theorems. 2. Karnaugh map (K-map) — graphical, up to 4 (5–6 with care) variables. 3. Tabulation / Quine–McCluskey method — algorithmic, works for any number of variables, needs a computer beyond 5–6. 4. NAND/NOR-only transformation and map-entered variables; and, in practice, minimisation by programmable logic (PLA mapping) or ESPRESSO-style heuristics.

Method 1 — algebra: `F = AB + AB′ = A(B + B′) = A·1 = A` (distributive law, then complement law).

Method 2 — K-map (two variables, rows A, columns B):

```
          B'     B
   A'    |  0     0
   A     |  1     1    <- AB' is the A,B' cell, AB is the A,B cell
```

The two 1s are adjacent and form one group of 2, which is the whole **A** row. B takes both values inside the group, so B drops out → **F = A**.

Method 3 — truth table (write it if you have time): A=1 gives F=1 in both B rows; A=0 gives F=0 in both.

Machine verification: over 2 variables the minterms of AB + AB′ = [2, 3] and the minterms of A = [2, 3] — identical → **True**. (Over 4 variables A,B,C,D the same two terms cover [8,9,10,11,12,13,14,15], which is exactly what A=1 gives there, so the check passes in any variable space.)

3.12 Q.4 (B) [6] Construct a D flip-flop using a JK flip-flop; write the D FF truth table

D flip-flop characteristic: **$Q^+ = D$** — the next state simply follows the input on the clock edge.

Truth table / characteristic table of a D FF:

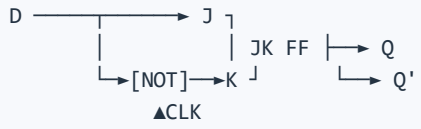
D	Q(t)	Q(t+1)	Action
0	0	0	Reset (stored 0)
0	1	0	Reset
1	0	1	Set
1	1	1	Set (transparent to D)

Excitation table (what the input must be to make the transition $Q \to Q^+$): $D = Q^+$.

To build it from a JK FF, match the two required JK rows to the four D rows:

```
D=0  -> Q+ = 0  -> J=0, K=1
D=1  -> Q+ = 1  -> J=1, K=0
=> J = D      and     K = D'
```

Wiring: feed D to J directly and through one inverter to K; clock to the FF's CLK; preset/clear left open.



Engine check: with J=D and K=D′, `Q⁺ = J·Q′ + K′·Q` equals D for both Q=0 and Q=1 → **verified: True**.

3.13 Q.4 (C) [10] JK flip-flop in detail; race-around condition and its solution; master-slave JK

JK FF — the improved SR FF with the forbidden state removed. Inputs J, K, clock; outputs Q, Q′.

J	K	Action	Characteristic equation
0	0	No change (hold)	$Q^+ = Q$
0	1	Reset	$Q^+ = 0$
1	0	Set	$Q^+ = 1$
1	1	Toggle	$Q^+ = Q'$

$Q^+ = J{\cdot}Q' + K'{\cdot}Q$ (verified for all 8 rows of J, K, Q — the engine confirms the table). Excitation table (needed for counter design):

$Q \to Q^+$	J	K
0→0	0	X
0→1	1	X
1→0	X	1
1→1	X	0

Race-around condition. In level-triggered JK with J=K=1, the output toggles; because the new output is fed back to the input AND gates while the clock is still high, it toggles again, and again — the final state after one clock pulse depends on how many toggles fit inside the pulse width, so it is unpredictable. It occurs when: (i)

the FF is level triggered, (ii) J=K=1, and (iii) the clock high time t_p satisfies **t_p > Δt** (propagation delay). Fix by making the pulse shorter than the delay is impractical, hence:

Solution 1 — edge triggering: respond only to the rising/falling edge; the feedback sees one instantaneous edge, so at most one toggle per clock. **Solution 2 — master-slave (pulse) configuration**, which is the diagram asked for:

```
        ┌── MSATER: two gated SR latches, opposite phases ──────────────┐
J ──►[AND]┐                                      ┌─[AND]◄── K'  (slave)
Q' ──►[AND]┘ MS ├──Q_M──► slave inputs ◄──┐      └─[AND]◄── Q    (slave)
K ──►[AND]┐                ┌──Q_M'──►[AND]─┘
Q ──►[AND]┘                └──Q_S──► feedback to master AND gates
CLK ──┬──►(via inverter)──► slave enable
      └──────────────────────► master enable
```

Operation: while **CLK = 0** the master is isolated and the slave tracks the master; when CLK goes **1** the master samples J and K (the slave is cut off); as CLK returns to **0** the master closes and the slave transfers the master's state to the outputs. Exactly **one** output change per clock pulse, so the race-around is eliminated because the feedback path is broken for half of every cycle. Timing to draw: CLK, Master-internal Q, slave Q, showing the slave changing only on the falling edge.

3.14 Q.4 (A OR) [4] Draw and explain the Sample and Hold circuit

Purpose: an ADC needs its input to be *constant* during conversion, but the signal is changing. A sample-and-hold (S/H) captures the instantaneous value and holds it flat for the conversion time.

Two states: **Sampling (track)** — switch closed, the capacitor charges to v_in through R_on, so v_out follows v_in; **Hold** — switch open, the capacitor is isolated and v_out stays at the last captured value (ideal forever, real circuits droop at I_leak/C).

```
       R_on            C              buffer (gain = 1)
v_in ──/ ──┤──┬──────────┴──────────┬──►[ + ]──┬──► v_out
     (switch)│                     │    [ - ]  │
  hold cmd ──┘ (FET gate)          └───────────┘
```

Key specs: **aperture time** (the uncertain interval over which the sample is actually taken), **acquisition/settling time** (how fast it tracks in sample mode), **hold-mode droop** (dV/dt leakage), and feed-through. The capacitor is the heart of the circuit — it stores the sampled level. Applications: front end of every ADC, peak detectors, data distribution, and analog multiplexing.

3.15 Q.4 (B OR) [6] Number of 16×4 chips needed for a 16×8 memory; design it

```
Chips required = total bits / bits per chip = (16 × 8) / (16 × 4) = 2
```

Only the **word length** must double (4 bits → 8 bits), the number of words stays 16, so this is **bit (organizational) extension**, not word extension:

```
                    ┌──────────────┐  address A3..A0 (4 lines) to BOTH chips
16 x 8 memory  =    │  16x4 chip 1 │  -> data D3..D0  (low nibble)
                    ├──────────────┤
                    │  16x4 chip 2 │  -> data D7..D4  (high nibble)
                    └──────────────┘  one common /WE, common /CS
```

Both chips share the same 4-line address decoder and the same read/write control; their output buffers are enabled only when /CS = 0 and for read /OE low; chip 1 drives the low four data lines, chip 2 the high four. Address lines needed: $\log_2 16 = 4$; data lines = **8**; each chip contributes 4. (If instead a 2048×8 memory were

asked from 256×8 chips the answer would be 8 chips with a 3-line 3:8 decoder for chip-select — word extension. State that contrast in one line to earn the reasoning mark.)

3.16 Q.4 (C OR) [10] 4-bit binary magnitude comparator with equations and logic diagram

Compare A = $A_3A_2A_1A_0$ against B = $B_3B_2B_1B_0$; three outputs: **A>B (AGT)**, **A=B (AeqB)**, **A<B (ALT)**. First define equality of one bit pair (an XNOR):

```
x_i = A_i ⊙ B_i = A_i·B_i + A_i'·B_i'
```

Then the higher-order bits dominate:

```
A>B = A3·B3' + x3·A2·B2' + x3·x2·A1·B1' + x3·x2·x1·A0·B0'
A<B = A3'·B3  + x3·A2'·B2  + x3·x2·A1'·B1  + x3·x2·x1·A0'·B0
A=B = x3·x2·x1·x0
```

Read it as: "the most significant pair that differs decides; if none differ, they are equal."

Machine verification: all **256** (A,B) pairs were evaluated against real integer comparison — the three equations match every pair → **True**.

Diagram to draw: four XNOR gates producing x_3…x_0, then four AND gates of increasing input count feeding one OR gate for A>B; a mirrored network for A<B; one 4-input AND for A=B. Show the IC 7485 as the standard 4-bit comparator chip and note that two 7485s cascade (with the three expansion inputs) for 8 bits.

3.17 Q.5 (A) [4] State and prove De Morgan's theorems using truth tables

Theorems: `(A·B)' = A' + B'` and `(A + B)' = A'·B'`.

Proof 1 — by enumeration:

A	B	A·B		A′	B′	
0	0	0	**1**	1	1	**1**
0	1	0	**1**	1	0	**1**
1	0	0	**1**	0	1	**1**
1	1	1	**0**	0	0	**0**

The two bold columns are identical → the theorem is proved.

A	B	A+B		
0	0	0	**1**	**1**
0	1	1	**0**	**0**
1	0	1	**0**	**0**
1	1	1	**0**	**0**

Engine result: both identities evaluate True over all four rows. General form to quote: complement of a *product* = *sum* of the complements, and vice-versa; extended to n variables, `(A B C …)' = A'+B'+C'+…`. The practical use is NAND/NOR-only implementation — every Boolean function can be drawn with one gate type.

3.18 Q.5 (B) [6] Construct a 3-bit asynchronous UP counter with preset and clear using JK FFs

Asynchronous (ripple): the external clock drives only the first FF; each FF's output clocks the next stage. All JK FFs are tied as toggle FFs (**J = K = 1**), so every active edge divides the frequency by 2.

```
CLK ──►[FF0] Q0 ──►[FF1] Q1 ──►[FF2] Q2      (Q0 = LSB, Q2 = MSB)
        J=K=1       J=K=1       J=K=1
        PRE, CLR    PRE, CLR    PRE, CLR   (all tied together)
```

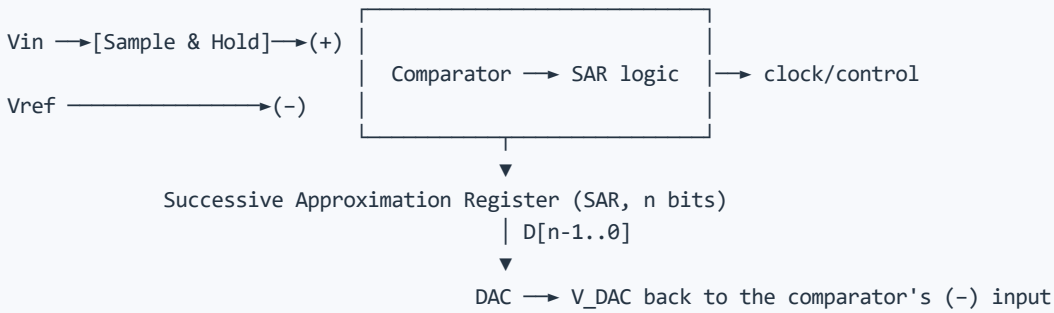
Because the FFs are **negative edge triggered**, Q0's falling edge starts FF1, and so on — the count rises. Sequence (verified by simulation): 000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000, i.e. mod-8. Timing diagram: Q0 has half the clock frequency, Q1 a quarter, Q2 an eighth; each transition of a higher bit is delayed by one FF propagation time behind the lower bit → **total worst-case delay = 3 × t_PD**, which is the asynchronous counter's one real weakness (state glitches at the ripple instant).

Preset and clear: async active-LOW inputs, so one **CLR** line wired to all three `/CLR` pins forces 000, and one **PRE** line wired to all `/PRE` pins forces 111. Add a two-input decoder if a mid-count preset value is wanted; for a modulo-N counter, decode count N and feed the reset (e.g. mod-6 = decode 110 → clear: 0,1,2,3,4,5, then back to 0 — verified).

3.19 Q.5 (C) [10] Successive-approximation ADC: principle plus circuit

Principle — binary search of the input range. The register is stepped through the bits, most significant first: set the bit, let the DAC produce that trial voltage, compare with V_in; keep the bit if the DAC output is still ≤ V_in, otherwise clear it. After n clock cycles the closest n-bit code is found, in exactly n comparisons ($2^n/2$ guesses saved).

Five blocks (draw all five inside the loop):



Operation trace for a 4-bit converter with V_ref = 5 V (step = 5/16 = 0.3125 V) and V_in = 3.2 V — the machine actually ran this loop:

Step	Bit tried	Code	DAC output	Keep?
1	b3	1000 (8)	2.5000 V	≤ 3.2 → keep
2	b2	1100 (12)	3.7500 V	> 3.2 → clear
3	b1	1010 (10)	3.1250 V	≤ 3.2 → keep
4	b0	1011 (11)	3.4375 V	> 3.2 → clear

Final code = 1010 (10) → 3.1250 V, quantisation error 3.2 − 3.125 = **0.075 V** (< 1 LSB ✓). Advantages: fast (fixed n cycles), compact, no integrator. Disadvantage: needs a stable input during conversion, hence the S/H front end. Conversion time = n × T_CLK (+ the S/H acquisition time); a 12-bit SAR at 1 MHz takes about 12 µs. Applications: general-purpose data acquisition, the standard "embedded ADC" (ADC0804, and every microcontroller ADC).

3.20 Q.5 (A OR) [4] Internal organisation of a 16×4 memory chip

A 16×4 chip stores 16 words of 4 bits = 64 cells.

```
A3 A2 A1 A0 ──►[ 2:4 / 4-line decoder ]──► 16 word lines (row select)
                                                  │
                      ┌── 64 cell matrix ─────────┘
                      │   each row = 4 cells (4 bits), each cell = 1 flip-flop (SRAM)
                      ▼
word line ──►[sense/write drivers]──► I/O buffers x4 ──► D3 D2 D1 D0 (bidirectional)
control: /CS (chip select), /OE (output enable, read), /WE (write enable)
```

Pins: 4 address + 4 data (shared in/out) + 3 control (+Vcc, GND) = far fewer pins than 64 cells because the matrix is addressed, and the four data lines are multiplexed onto the four bit-columns of the selected row. Read = decoder selects the row, /CS and /OE low, drivers place the 4 bits on the bus. Write = same row selected, /WE pulsed, the bus bits are driven into the cells. Scaling note that usually comes with the next part of the question: to make 16×8 from two such chips see §3.15; 2048×8 needs eight 256×8 chips plus a 3-line 3:8 decoder on the high address lines for chip-select.

3.21 Q.5 (B OR) [6] Synchronous counter 0, 1, 3, 4, 5, 7 and repeat, using T flip-flops

Three states bits A B C (A = MSB) cover values 0…7; **2 and 6 never occur → don't-care** in the excitation map, which is what makes the equations collapse.

Present→next state table with the T excitation ($T = Q \oplus Q^{+}$):

PS (ABC)	Decimal	NS	T_A T_B T_C
000	0	001	0 0 1
001	1	011	0 1 0
010	2	—	X X X
011	3	100	1 1 1
100	4	101	0 0 1
101	5	111	0 1 0
110	6	—	X X X
111	7	000	1 1 1

Minimised input equations (each exhaustively verified against the table above):

```
T_A = B                  (Σm(3,7) + d(2,6))
T_B = C                  (Σm(1,3,5,7) + d(2,6))
T_C = B + C'             (Σm(0,3,4,7) + d(2,6))
```

Full solution steps to write: (1) state diagram of the cycle; (2) state/excitation table; (3) three K-maps with Xs at 2 and 6; (4) logic diagram — three T FFs on a common clock, T_A fed from B, T_B from C, T_C from an OR of B and C′; (5) **self-starting check**. The machine simulated the design from every start state:

```
from 0: 0 -> 1 -> 3 -> 4 -> 5 -> 7 -> 0       exactly the required cycle (verified True)
from 2: 2 -> 3 -> 4 -> 5 -> 7 -> 0 -> 1 -> 3  enters the cycle in 1 clock
from 6: 6 -> 7 -> 0                           enters the cycle in 2 clocks
```

Because both unused states fall into the loop, the counter is **self-starting** — always state that sentence, it is a marking point.

3.22 Q.5 (C OR) [10] R-2R ladder DAC: circuit, operation, advantage

Only two resistor values — R and 2R — no matter how many bits. That is the whole selling point.

```
b3 ──2R──┬──b2──2R──┬──b1──2R──┬──b0──R──┐     (each bit switched to Vref or to 0 V)
         │          │          │         │
        2R         2R         2R        └──► to op-amp virtual ground (summing node)
         │          │          │
        ...        ...        ...
  ladder terminates in 2R to ground at the far end
```

Every node looking right from a switch sees exactly **2R**, so the network is a binary voltage divider: the bit at position i contributes `(Vref · D_i)/2^(i+1)` and the total at the summing junction is

```
V_A = Vref · (b3·2³ + b2·2² + b1·2¹ + b0·2⁰) / 2⁴  =  Vref · D / 2ⁿ
```

Table for a 4-bit ladder at V_ref = 5 V (computed, not copied):

D (decimal)	Binary	V_out
0	0000	0.0000 V
1	0001	0.3125 V
5	0101	1.5625 V
10	1010	3.1250 V
15	1111	4.6875 V

Resolution = V_ref/2^n = 5/16 = 0.3125 V per step; full-scale = V_ref·(2^n−1)/2^n = 4.6875 V. **Advantages over the weighted-resistor DAC:** (i) only two resistor values instead of n precisely-scaled ones (R, R/2, R/4, R/8 …), so an 8-bit converter does not need resistors ranging from R to 128R; (ii) easy and cheap to fabricate in IC form, tolerance is far easier to hold below 0.5%; (iii) the ladder's output impedance is constant at 2R whatever the code, so the switching errors don't scale with the pattern; (iv) accuracy is set by the resistor *ratio*, not the absolute value. Draw the ladder feeding the inverting input of an op-amp with R_f so that V_out = −I_total·R_f; label V_ref, the switches (usually implemented as a current-steering differential pair per bit), 2R terminations, and the binary inputs b_0…b_3 with b_0 marked LSB.

4. Winter 2024 (27/12/2024) — Q.1 to Q.5 fully solved

> The 2024 paper is the "definition + design" paper: Q.1 is ten one-liners, and Q.2–Q.5 follow the 2025 pattern (4 + 6 + 10, with an internal OR on the 10-mark part). Several questions are word-for-word the same as a 2025 question or an assignment question — those entries point at the worked answer in section 2 or 3 instead of repeating it.

4.1 Q.1 (A) [2] Convert the decimal number 25 into its binary equivalent

Repeated division by 2, remainders read **bottom-up**:

```
25 ÷ 2 = 12 r 1   <- LSB
12 ÷ 2 =  6 r 0
 6 ÷ 2 =  3 r 0
 3 ÷ 2 =  1 r 1
 1 ÷ 2 =  0 r 1   <- MSB
```

Answer: $(25)_{10} = (11001)_2$

Weight check (this line is what earns the mark): 16 + 8 + 1 = 25 ✓ Faster route: 25 = 16 + 8 + 1 → 1 at 2^4, 1 at 2^3, 0 at 2^2, 0 at 2^1, 1 at 2^0.

4.2 Q.1 (B) [2] Define the term "universal gate"

A **universal gate** is a gate whose repetitions alone can implement *every* Boolean function — the three basic operations NOT, AND and OR can all be built from it, so no other gate type is needed. The two universal gates are **NAND** and **NOR**.

Everything else (AND, OR, NOT, XOR, XNOR, buffer) then becomes a network of that one gate type, which is why IC families are manufactured as NAND/NOR arrays. Constructions: **§4.11** (NOR-only) and **§2.2** (XOR from four NANDs).

4.3 Q.1 (C) [2] What is the main characteristic of tristate gates?

The output has **three states instead of two**: logic 0, logic 1, and **high impedance (Z)** — the gate is electrically disconnected from its output line.



Consequence: many tristate outputs may be tied together to form a **shared bus**, because only one driver is enabled at a time while the rest present Z. This is the idea behind bus transceivers (74245), memory data lines, and the tri-state outputs of every microprocessor.

4.4 Q.1 (D) [2] Write the Boolean expression for the AND gate

Y = A · B (two-input); for n inputs `Y = A·B·C·…` — the output is 1 **only when every input is 1**.

A	B	Y = A·B
0	0	0
0	1	0
1	0	0
1	1	1

Extras the examiner expects in one line: it is logical **multiplication**, the symbol is the flat-ended D shape, TTL parts are 7408 (2-input), 7411 (3-input), 7421 (4-input); in positive logic a diode AND is the same function.

4.5 Q.1 (E) [2] Difference between SOP (Sum of Products) and POS (Product of Sums)

	SOP (Sum of Products)	POS (Product of Sums)
Form	OR of AND terms: `F = AB + C'D`	AND of OR terms: `F = (A+B)(A'+C)`
Built from	**minterms** — rows where F = 1	**maxterms** — rows where F = 0
Notation	`F = Σm(...)`	`F = ΠM(...)`
Literal inside a term	variable is **uncomplemented where the row has 1**	variable is **complemented where the row has 1**
Implementation	AND plane → OR plane	OR plane → AND plane
Relationship	the two forms are duals: De Morgan converts SOP ↔ POS	

Worked example from this syllabus (**§2.8**): for `F = (xy+z)(y+xz)` the truth table gives `F = Σm(3,5,6,7) = x'yz + xy'z + xyz' + xyz` (SOP) and `F = ΠM(0,1,2,4) = (x+y+z)(x+y+z')(x+y'+z)(x'+y+z)` (POS).

4.6 Q.1 (F) [2] Name any two combinational circuits used in digital systems

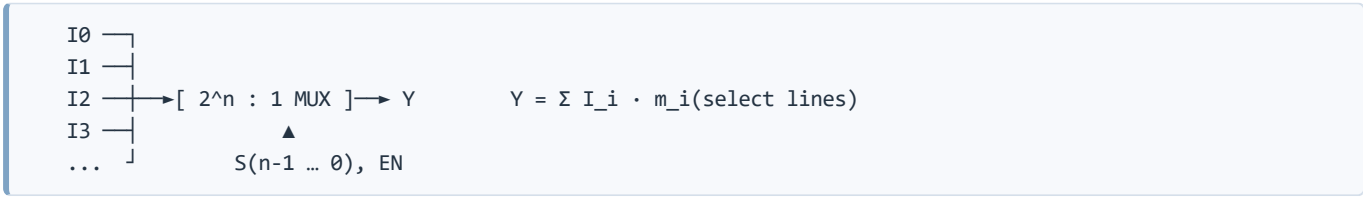
Adder (half / full adder) and **multiplexer**. Any two of these are also accepted: decoder, encoder, priority encoder, demultiplexer, magnitude comparator, ALU, subtractor.

One sentence each (this is what converts a 1 into a 2): * *Half adder* — adds two single bits and produces Sum and Carry: `S = x ⊕ y`, `C = x·y` (**§3.3**). * *Multiplexer* — selects one of 2^n data inputs onto one output using n select lines; a data selector (**§4.7**).

Definition to quote: in a combinational circuit the output depends **only on the present combination of inputs**, with no memory of past inputs and no clock.

4.7 Q.1 (G) [2] What is the function of a multiplexer?

A multiplexer (**MUX**) is a **data selector**: it routes **one of many input lines to a single output line**, chosen by the binary value on its select lines. `2ⁿ` data inputs need `n` select lines; most MUXes also carry a strobe/enable.



Two uses to mention: (i) moving data from several sources over one line or bus, (ii) **realising any Boolean function** — put the function's variables on the select lines and hard-wire the residual functions on the data inputs (fully worked in **§3.2**). IC examples: 74151 (8:1), 74153 (dual 4:1), 74157 (quad 2:1).

4.8 Q.1 (H) [2] What is a flip-flop? Name two types of flip-flops

A **flip-flop** is a bistable sequential element (two stable states) built from cross-coupled gates; it stores **one bit**, and its output changes only under control of a clock edge, so it keeps its value between clock events.

Any two types: **SR flip-flop** and **JK flip-flop** (D and T are also accepted). The JK FF is called *universal* because it has no forbidden input and can be wired to behave as SR, D or T.

Type	Characteristic equation	Forbidden input
SR	`Q+ = S + R'Q`	S = R = 1
JK	`Q+ = JQ' + K'Q`	none
D	`Q+ = D`	none
T	`Q+ = T ⊕ Q`	none

4.9 Q.1 (I) [2] Define a shift register and mention one of its applications

A **shift register** is a cascade of flip-flops sharing one clock in which, on every clock pulse, the content of each stage moves ("shifts") into the next stage, so data is moved serially through the register instead of being loaded and stored in parallel. An n-bit shift register needs **n flip-flops and n clock pulses** to accept or emit a serial word.

Application to write: serial ↔ parallel conversion — e.g. a UART receiving one bit per clock and presenting a whole byte on parallel outputs (SISO → SIPO), or generating an exact delay of n clock periods.

The four modes (SISO, SIPO, PISO, PIPO) with diagrams are in **§3.4**; the bidirectional case is **§4.15**.

4.10 Q.1 (J) [2] What is the purpose of a sample-and-hold circuit in A/D conversion?

An ADC needs a **steady input** while it measures, but a conversion takes many clock periods. The sample-and-hold circuit **captures the instantaneous input voltage on a capacitor and holds that value constant at the ADC input for the whole conversion**, presenting a flat snapshot instead of a moving waveform.

```
sample mode : switch closed -> C charges to v_in          (acquisition time)
hold    mode: switch open   -> C keeps the last value, the ADC reads a constant voltage
```

Without it a fast-changing input would be quantised to a mixture of several instants — aperture uncertainty, which shows up as conversion error and missing codes. The full circuit (op-amp, switch, hold capacitor, buffer) and its working are in **§3.14**.

4.11 Q.2 (A) [4] "Only NOR gates can be utilised for AND, OR and NOT gates" — why?

The sentence is the claim that **NOR is a universal gate**: each of the three basic operators can be expressed using NOR alone, so no other gate type is required.

```
NOT  :  A' = (A + A)'              one NOR with the inputs tied together
OR   :  A + B = ((A + B)')'        two NOR: a NOR followed by a NOR used as an inverter
AND  :  A·B = (A' + B')'           De Morgan — invert each input, then NOR them
```

Gate count when only 2-input NORs are allowed: NOT = 1, OR = 2, AND = 3 (two inverters into one NOR).

A	B	A NOR B	NOT A = (A+A)′	OR = ((A+B)′)′	AND = (A′+B′)′
0	0	1	1	0	0
0	1	0	1	1	0
1	0	0	0	1	0
1	1	0	0	1	1

Machine verification (`new_exam/verification3.txt`, block 4.11): for all four (A,B) combinations the NOR-built NOT / OR / AND reproduce the basic gates exactly → **True**; the same check with NAND only → **True**.

So the honest full answer is: **both NAND and NOR are universal** — the paper's statement is correct but not exclusive. Close with why it matters: an IC built from one gate type is cheaper to fabricate, and unused NOR/NAND gates on a package can be reconnected to obtain the extra functions.

4.12 Q.2 (B) [6] Simplify in SOP using a K-map: F = BD + A′C′D′ + A′B + A′CD′ + AB′D′

Step 1 — expand each given term into cells (order A B C D, A = MSB). Do this carefully; a term covers 2^(4 − literals) cells:

Term	Fixed bits	Cells covered
BD	B=1, D=1	5, 7, 13, 15
A′C′D′	A=0, C=0, D=0	0, 4
A′B	A=0, B=1	4, 5, 6, 7
A′CD′	A=0, C=1, D=0	2, 6
AB′D′	A=1, B=0, D=0	8, 10

Union → **on-set = Σm(0, 2, 4, 5, 6, 7, 8, 10, 13, 15)** (this is exactly what the engine returns).

Step 2 — plot the 1s. Rows = AB, columns = CD, both in Gray order:

```
              CD
          00    01    11    10
        ┌─────┬─────┬─────┬─────┐
AB  00  │  1  │  0  │  0  │  1  │   m0, m1, m3, m2
    01  │  1  │  1  │  1  │  1  │   m4, m5, m7, m6
    11  │  0  │  1  │  1  │  0  │   m12, m13, m15, m14
    10  │  1  │  0  │  0  │  1  │   m8, m9, m11, m10
        └─────┴─────┴─────┴─────┘
```

Step 3 — circle the three prime groups:

Group	Cells	Group reads	Term
1	m5, m7, m13, m15	B = 1 and D = 1 in all four	**BD**
2	m4, m5, m6, m7	A = 0 and B = 1	**A'B**
3	m0, m2, m8, m10 (the four corners of columns 00/10 in rows 00/10)	B = 0 and D = 0	**B'D'**

Every 1 is inside at least one group, and no group can be dropped.

Answer: F = BD + A'B + B'D' — 3 terms, 6 literals.

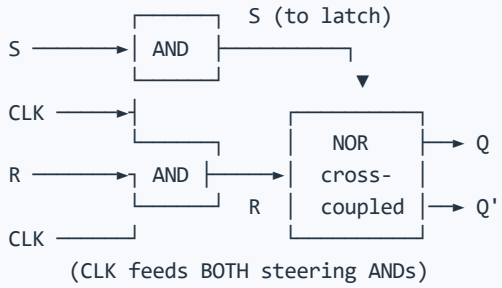
Machine verification (`verification.txt`, W2024 Q.2(B) block): minimal SOP `= BD + A'B + B'D'`, cover verified **True**, exhaustive search over every prime-implicant subset says **no 2-term or 5-literal cover exists**.

Marks note: the original expression has 5 terms and 13 literals; the answer has 3 terms and 6 literals. Write both counts — "simplified" is worth a mark only if you show what it was simplified *from*.

4.13 Q.2 (C) [10] Explain the S-R clocked flip-flop

*(OR branch — internal organisation of a 16×4 memory chip: solved in **§3.20**.)*

An asynchronous SR **latch** changes state the instant its inputs change. A **clocked (gated) SR flip-flop** puts two steering AND gates in front of the latch so that it only listens while the clock (or enable) pulse is active.



Working: 1. **CLK = 0** — both AND gates output 0 whatever S and R do, so the latch is driven with (0,0) and **holds** its previous state; the inputs are ignored. Verified: E = 0 always drives the latch inputs to (0,0) → **True**. 2. **CLK = 1** — the steering gates pass the inputs straight through, so `S=1,R=0` sets, `S=0,R=1` resets and `S=0,R=0` holds. Verified: E = 1 passes every legal pair unchanged → **True**. 3. **CLK = 1 with S = R = 1** — the latch is driven with S = R = 1, both NOR outputs fall to 0, hence Q = $\bar{Q}$ = 0 (not a complementary pair). When CLK returns to 0 the latch sees (0,0) and settles into whichever gate wins first — the next state is **undetermined**. This input pair is **forbidden**.

S	R	CLK	Q(n+1)	Name
0	0	1 (or ↑)	Q(n)	memory / no change
0	1	1 (or ↑)	0	reset
1	0	1 (or ↑)	1	set
1	1	1 (or ↑)	X	**not allowed**
X	X	0	Q(n)	inhibited

Points that carry the remaining marks: * it is **level-sensitive (transparent)**: while CLK stays 1 the output follows the inputs, which is why edge-triggered and master-slave versions exist (**§3.13**); * **excitation table** — `Q→Q+` : 0→0 needs S=0,R=X; 0→1 needs S=1,R=X; 1→0 needs S=X,R=1; 1→1 needs S=X,R=0; * characteristic equation `Q+ = S + R'Q` with the constraint `S·R = 0` (proved in **§4.25** where this FF is built out of a D FF); * one clocked FF stores exactly **1 bit**, so a 16×4 chip needs 64 of them (**§3.20**); * draw the input/output waveform: S and R pulses that arrive while CLK is low must produce **no** change.

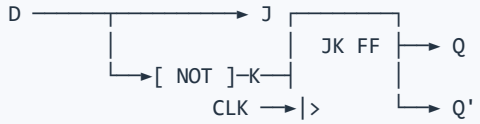
4.14 Q.3 (A) [4] Design a D flip-flop using a J-K flip-flop

Full derivation is in **§3.12**; the short version to reproduce in the paper:

`Q+ = JQ' + K'Q` (JK) must equal `Q+ = D` (D FF). Compare them over the four (D, Q) rows:

D	Q	Q+ required	J	K
0	0	0	0	X
0	1	0	X	1
1	0	1	1	X
1	1	1	X	0

Choosing the don't-cares to agree gives **J = D** and **K = D'** — one inverter and one wire.



Machine verification (`verification3.txt`, block 4.14): `J = D, K = D'` forces the next state to equal D for **both** present states → **True**. Add one line on the advantage: the D FF has a single data input and no forbidden combination, which is why registers, shift registers and memory cells are built from D flip-flops.

4.15 Q.3 (C) [10] Draw and explain the logic diagram of a 4-bit bidirectional shift register

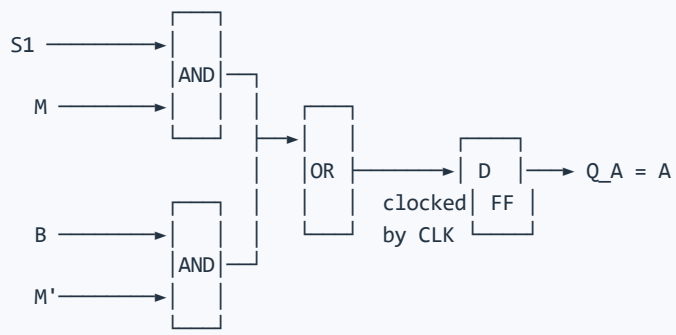
A bidirectional shift register can move its content **left or right under a mode control M**. Each stage is a D flip-flop whose data input is chosen by a pair of AND gates and an OR gate (equivalently a 2:1 multiplexer) — this is the 74194 arrangement.

Convention used below: **A = MSB, D = LSB**, `M = 1` shifts right (towards D, new bit enters at A from the right serial input S1), `M = 0` shifts left (new bit enters at D from the left serial input S0).

Input equations:

```
D_A = M·S1 + M'·B
D_B = M·A  + M'·C
D_C = M·B  + M'·D
D_D = M·C  + M'·S0
```

One stage of the logic diagram (draw all four identically, with the neighbour taps swapped):



(the B tap comes from the next stage on the right; the same block, shifted, builds D_B, D_C, D_D)

Operation traced with `ABCD = 1011` and both serial inputs S1 = S0 = 1 — the trace is computed in `new_exam/verification2.txt`, "bidirectional" block:

Mode	Movement	Next state	Result
M = 1	right: A←S1=1, B←A, C←B, D←C	1 1 0 1	**1101**
M = 0	left: A←B, B←C, C←D, D←S0=1	0 1 1 1	**0111**

Points that carry the 10 marks: * one common clock drives all four FFs, so they change together — a **synchronous** shift register; * parallel load is two extra AND-OR paths; the 74194 mode pins are `S1 S0 = 00 hold, 01 shift right, 10 shift left, 11 parallel-load P0…P3`; * **access time**: a serial word needs n clock pulses, a parallel word needs one; * speed: only one FF + gate delay per shift, so the maximum clock frequency is far better than a ripple counter's cumulative delay; * Q outputs (and the serial inputs) are available at both ends, so **any number of bits can be cascaded** by connecting the last stage's Q to the next package's serial input; * applications: serial↔parallel conversion, exact delay lines (n·T_CLK), LED chaser / bar-graph display, serial arithmetic, and pseudo-random sequences when an XOR of selected taps feeds the serial input; * two special closed forms worth naming: **ring counter** (last Q drives the first D, modulus n) and **Johnson counter** (last $\bar{Q}$ drives the first D, modulus 2n).

4.16 Q.3 (A OR) [4] Design a combinational circuit for F_1, F_2, F_3 using a decoder and external gates

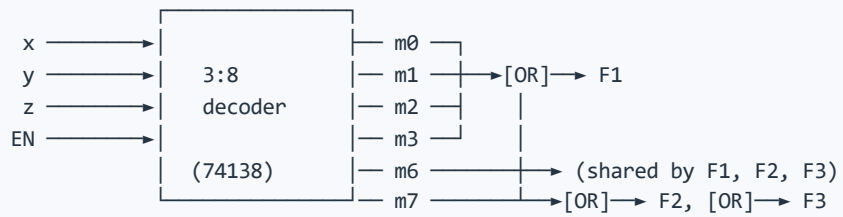
```
F1 = x'y' + xyz'        F2 = x' + y        F3 = xy + x'y'
```

A 3:8 decoder with inputs `A2 = x, A1 = y, A0 = z` produces all eight minterms m_0…m_7, so each function only needs the minterms OR-ed together. Expanding each term (x y z, x = MSB):

Term	Fixed bits	Minterms
x′y′	x=0, y=0, z free	m0, m1
xyz′	x=1, y=1, z=0	m6
x′	x=0, y z free	m0, m1, m2, m3
y	y=1, x z free	m2, m3, m6, m7
xy	x=1, y=1, z free	m6, m7
x′y′	as above	m0, m1

Therefore (exactly what `verification.txt` reports for W2024 Q.3(OR)(A)):

```
F1 = Σm(0, 1, 6)          = OR(m0, m1, m6)
F2 = Σm(0, 1, 2, 3, 6, 7) = OR(m0, m1, m2, m3, m6, m7)
F3 = Σm(0, 1, 6, 7)       = OR(m0, m1, m6, m7)
```



Draw three OR gates (3-input, 6-input, 4-input) feeding from the decoder outputs. Say the two sentences that earn the marks: a decoder generates **all 2^n minterms for free**, so any combinational function becomes a two-level "decoder + OR" realisation; no K-map minimisation is needed because the decoder *is* the AND plane. Mention 74138 (3:8, active-low outputs — then the OR gates become AND gates, or use one OR-inverter).

4.17 Q.3 (B OR) [6] Draw the truth table of a 2-bit digital comparator

Compare `A = A₁A₀` against `B = B₁B₀` (A_1, B_1 are the MSBs). Three outputs: **A>B**, **A=B**, **A<B**.

Equations (cascade form — the more significant pair decides first):

```
A>B = A₁B₁' + (A₁⊙B₁)·A₀B₀'
A=B = (A₁⊙B₁)·(A₀⊙B₀)
A<B = A₁'B₁ + (A₁⊙B₁)·A₀'B₀        where  X⊙Y = XY + X'Y'  (XNOR)
```

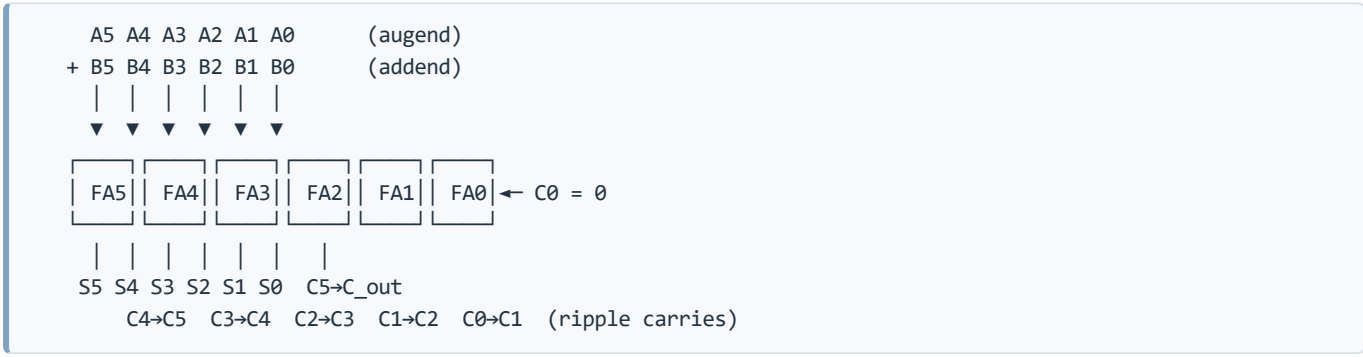
Full table — all 16 combinations, generated and checked against real integer comparison (`verification3.txt`, block 4.17 → **True**, and "exactly one output is 1 in every row" → **True**):

A_1 A_0	B_1 B_0	A	B	A>B	A=B	A<B
0 0	0 0	0	0	0	1	0
0 0	0 1	0	1	0	0	1
0 0	1 0	0	2	0	0	1
0 0	1 1	0	3	0	0	1
0 1	0 0	1	0	1	0	0
0 1	0 1	1	1	0	1	0
0 1	1 0	1	2	0	0	1
0 1	1 1	1	3	0	0	1
1 0	0 0	2	0	1	0	0
1 0	0 1	2	1	1	0	0
1 0	1 0	2	2	0	1	0
1 0	1 1	2	3	0	0	1
1 1	0 0	3	0	1	0	0
1 1	0 1	3	1	1	0	0
1 1	1 0	3	2	1	0	0
1 1	1 1	3	3	0	1	0

Finish the answer with: four XOR/XNOR gates for the bit equalities, AND/OR network for the two inequalities, the standard 4-bit part **7485**, and the note that 7485 has cascade inputs so two of them make an 8-bit comparator (see **§3.16** for the 4-bit equations and diagram).

4.18 Q.3 (C OR) [10] Explain a 6-bit binary parallel adder with the necessary diagrams

A **parallel adder** adds two n-bit words in one operation by using **n full adders connected back-to-back**, the carry-out of each stage feeding the carry-in of the next more significant stage.



Each full adder (proved in **§3.3**):

```
S_i   = A_i ⊕ B_i ⊕ C_i
C_i+1 = A_i·B_i + C_i·(A_i ⊕ B_i)
```

For six bits: six Sum equations, six carry equations, and the 7th carry C_6 is the overflow bit — so a 6-bit adder produces a **7-bit** result.

Machine verification (`verification3.txt`, block 4.18): a 6-bit ripple adder built exactly this way was evaluated against integer addition for **all 4096 (A,B) pairs** → **True**. Worked example quoted by the checker: 13 + 7 = 20 → `0010100` in 7 bits.

Diagrams and details to draw for the 10 marks: 1. the six-FA block diagram above; 2. one FA's gate-level circuit (two HAs + OR, or SOP form `C = AB + C(A⊕B)`); 3. the half adder that a 6-bit adder would need if FA_0 is replaced by a HA when $C_0 = 0$; 4. the **propagation delay** argument: the carry must ripple through all 6 stages, so the worst-case sum-bit delay is `6 · t_carry` (≈ 6 × 2 gate delays with 7483 slices) while each Sum needs only two XOR delays — this is the drawback to name; 5. the fix: **look-ahead carry** with `G_i = A_i·B_i`, `P_i = A_i + B_i`, so all carries are produced in parallel (worked in **§3.8**, verified over 512 combinations); 6. the IC reference: 7483 = 4-bit parallel adder; two 7483s cascade (C_out of the low slice to C_in of the high slice) for 8 bits; a 6-bit adder = one 4-bit slice + one 2-bit slice; 7. applications: ALU arithmetic, binary subtraction through 2's complement (`A + B' + 1`), BCD adders with a +6 correction network, magnitude comparison.

4.19 Q.4 (A) [4] Define: Figure of merit, Noise margin, Power dissipation

Power dissipation (P_D) — the power a gate draws from the supply, averaged over one complete output cycle (high and low). It is quoted in milliwatts per gate; lower P_D means more gates per chip and less cooling. Typical: TTL ≈ 10 mW, low-power TTL ≈ 1 mW, CMOS ≈ 0.001 mW static.

Noise margin — the largest noise voltage that may be added to an input without changing the output state. Two numbers per family:

```
V_NH = V_OH(min) − V_IH(min)
V_NL = V_IL(max) − V_OL(max)
```

For standard TTL: V_OH(min) = 2.7 V, V_IH(min) = 2.0 V → **V_NH = 0.7 V**; V_IL(max) = 0.8 V, V_OL(max) = 0.4 V → **V_NL = 0.4 V**. Both must be positive for valid operation.

Figure of merit (FoM) — the product of propagation delay and power dissipation; it is the energy per switching event and the fair way to compare families:

```
FoM = t_PD × P_D      units: ns × mW = pJ      (smaller is better)
```

Family	t_PD	P_D	FoM
TTL (74)	10 ns	10 mW	**100 pJ**
ECL	1 ns	25 mW	**25 pJ**
CMOS	10 ns	0.0075 mW	**0.075 pJ**

Machine verification: the arithmetic above is computed in `verification2.txt` (logic-family block), and the noise-margin substitutions are the ones quoted in **§2.5 → True**.

Add the closing line: CMOS wins the figure of merit by three orders of magnitude, which is why it dominates; ECL wins raw speed because its transistors never saturate.

4.20 Q.4 (B) [6] Simplify to the minimum number of literals: F(w,x,y,z) = Σ(0,1,2,4,5,6,8,9,12,13,14)

Eleven of the sixteen cells are 1, so plot it (w = MSB; rows wx, columns yz):

```
            yz
         00    01    11    10
      ┌─────┬─────┬─────┬─────┐
wx 00 │  1  │  1  │  0  │  1  │   m0, m1, m3, m2
   01 │  1  │  1  │  0  │  0  │   m4, m5, m7, m6
   11 │  1  │  1  │  0  │  0  │   m12, m13, m15, m14
   10 │  1  │  1  │  0  │  0  │   m8, m9, m11, m10
      └─────┴─────┴─────┴─────┘
```

Groups, largest first:

Group	Cells	Reading	Term
1	m0,1,4,5,8,9,12,13 — the eight cells with y = 0	y is 0 in all of them	**y′**
2	m4,6,12,14 — column yz = 10 rows… use rows 01/11 with z = 0	x = 1 and z = 0	**xz′**
3	m0,2,4,6 → after groups 1–2 the only uncovered 1 is m2 (0010)	w = 0 and z = 0	**w′z′**

Coverage check: y′ gives 0,1,4,5,8,9,12,13; xz′ adds 6,14; w′z′ adds 2 → all eleven 1s, and the three groups contain no 0 cell.

Answer: F = y′ + xz′ + w′z′ — 3 terms, 5 literals.

Machine verification (`verification.txt` W2024 Q.4(B), repeated in `verification3.txt` block 4.20): minimal SOP `= y' + xz' + w'z'`, cover verified **True**, exhaustive search confirms **no 4-literal cover exists**, and 11 of 16 cells is not reducible to 2 terms.

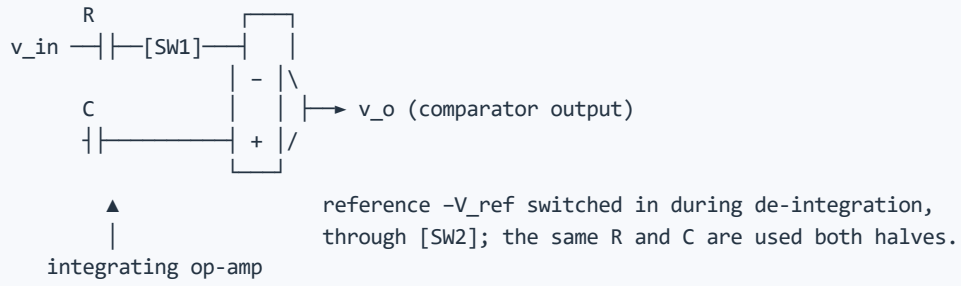
Exam tip: because the function is nearly all 1s, simplify the **zeros** instead — they are m3, m7, m11, m15 = the whole yz = 11 column, i.e. `F′ = yz`, giving `F = (yz)′` … which is **wrong** here (12 cells would then be 1, not 11). Reading a map upside-down is the most common lost mark in this question; verify by counting cells, as above.

4.21 Q.4 (C) [10] List the various A/D converters and explain any one

Types (write all of these — the "list" is a mark on its own): 1. **Flash / parallel comparator ADC** — $2^n - 1$ comparators, fastest, most hardware. 2. **Successive-approximation (SAR) ADC** — binary search with a DAC

in the feedback loop (worked in **§3.19**). 3. **Single-slope (linear-ramp) ADC**. 4. **Dual-slope (integrating-type) ADC** — explained below. 5. **Voltage-to-frequency (V/F) converting ADC**. 6. **Sigma-delta (Δ-Σ) / charge-balancing ADC**. 7. **Pipeline (sub-ranging) ADC**. 8. Counting type and ramp-and-ADC hybrids (the basic counter-ramp ADC).

Dual-slope ADC — circuit and working



Phase 1 — **auto-zero / integration of the unknown**: switch to v_in for a **fixed** time $T_1 = n_1 \cdot T_clk$, the counter counts from 0 to n_1 and the integrator output ramps to

$$V_{pk} = (1/RC) \int_0^{T1} v_{in}\, dt = v_{in} \cdot T_1 / (R \cdot C) \qquad \text{(for a constant } v_{in})$$

Phase 2 — **de-integration of the reference**: the input is switched to the fixed reference −V_ref and the integrator ramps back towards zero at a known constant rate. The counter keeps counting until the comparator detects zero crossing; that measured time is

$$T_2 = V_{pk} \cdot RC / V_{ref} = T_1 \cdot v_{in} / V_{ref} \quad \rightarrow \quad R \text{ and } C \text{ cancel completely}$$

so the digital result $N_2 = T_2/T_clk = n_1 \cdot v_{in} / V_{ref}$ is **independent of the values of R and C** — this is the single most important advantage to write, and why dual-slope converters are used in precision digital multimeters.

Worked numbers (computed in `verification3.txt`, block 4.21 — V_in = 3.2 V, V_ref = 5 V, T_1 = 100 ms at a 10 kHz clock, so n_1 = 1000 counts):

R	C	RC	V_pk at end of T_1	T_2	counts
10 kΩ	1.00 µF	0.0100 s	32.000 V	64.0 ms	**640**
22 kΩ	0.47 µF	0.0103 s	30.948 V	64.0 ms	**640**
100 kΩ	0.10 µF	0.0100 s	32.000 V	64.0 ms	**640**

The same reading from three very different RC products — the checker's line is `T2 = T1 * V_in / V_ref = 0.064 s -> 640 counts (independent of R and C)` → **True**.

Other properties to list: conversion time is $2 \cdot T_1$ (slowest of the common types), excellent **noise rejection** — an integration period equal to a multiple of the mains period rejects 50/60 Hz induced noise exactly, monotonic by construction, low part count, resolution limited by n_1 (full scale V_in = V_ref gives $T_2 = T_1$). Block diagram to draw: integrator → comparator → counter → control flip-flop/SAR switch logic, with the two switches on the integrator input.

*(If you are faster on the SAR description, write that instead — it is fully worked with a numeric trace in **§3.19**. Never write both in an answer sheet; one complete converter in detail is what the 10 marks ask.)*

4.22 Q.4 (A OR) [4] Short note on Read Only Memory (ROM)

ROM is a memory whose stored pattern is fixed (or changed only by a special procedure): the device provides address inputs and data outputs, and the CPU can only **read** it. Non-volatile, used for firmware, look-up tables,

boot code, character generators and function tables.

Structure — a ROM is exactly **a decoder plus a switch matrix plus output buffers**:

```
address lines (n) ──►[ n-to-2^n decoder ]──► word lines (2^n)
                                  │
                                  ▼
                     [ fuse / transistor array ]   <-- the stored pattern
                                  │
                                  ▼
                     [ output buffers ] ──► data lines (m)
```

Capacity is quoted as **words × bits per word**, so the total number of stored bits is $2^n \times m$. Machine-checked examples (`verification3.txt`, block 4.22):

Address lines	Words	Output bits/word	Total bits
3	8	8	**64**
4	16	5	**80**
5	32	32	**1024**

Types to name: **Mask ROM** (pattern set during fabrication), **PROM** (one-time programmable fuses), **EPROM** (UV-erase, floating gate), **EEPROM / E²PROM** (electrical erase, byte-wise), **Flash** (block erase, USB drives and SSDs).

The examiner's extra point: a ROM is a **combinational logic generator** — the decoder forms all minterms and the programmable OR plane selects them, so any function table can be wired by writing the truth table into the memory (a 3-to-8 decoder *is* an 8×3 ROM with the minterms hard-wired). That is the same idea as **PLA vs ROM**: PLA has a programmable AND *and* OR plane and only the needed product terms; ROM has a fixed AND plane (full decoder) and a programmable OR plane, so it is simpler but wastes terms.

4.23 Q.4 (B OR) [6] Compare static RAM and dynamic RAM

Aspect	SRAM	DRAM
Cell	6 transistors (two cross-coupled inverters + 2 access)	1 transistor + 1 capacitor
Data held by	latch feedback (voltage levels)	charge on a capacitor
Refresh	**not needed** while powered	**needed**, typically every 8–16 ms (row refresh)
Leakage	negligible	significant — charge leaks away
Access time	~10–25 ns	~50–100 ns
Cycle time	short	longer (refresh steals cycles)
Density	low (large cell area)	high (roughly 4–6× more bits per mm^2)
Power per bit	higher static power	lower device power, plus refresh power
Cost per bit	high	low
Peripheral complexity	simple	needs RAS/CAS address multiplexing + refresh counter + sense amps
Addressing	full address in one go	row address then column address on the same pins (fewer pins)
Sensing	level sensing	charge sharing with a pre-charged bit line, then rewrite
Used for	CPU caches L1/L2/L3, register files, buffer memory	main system memory
Volatility	volatile (both lose contents without power)	volatile

Three sentences that finish the answer: DRAM is read-destructive, so every read must be followed by a rewrite (the sense amplifier does this); SRAM is used where speed matters and DRAM where capacity and cost matter; a hybrid machine therefore puts SRAM cache in front of DRAM main memory. Refresh modes to name: **burst**, **distributed** and **hidden (asynchronous)** refresh.

4.24 Q.4 (C OR) [10] Full subtractor with truth table, and its design using a 3:8 decoder with OR gates

A **full subtractor** subtracts three bits: minuend x, subtrahend y and the incoming borrow b, giving **Difference D** and outgoing **Borrow B**.

Logic derivation: `D = 1` when an odd number of inputs are 1 → `D = x ⊕ y ⊕ b`. `B = 1` when the value being subtracted exceeds x → `B = x'y + x'b + yb`.

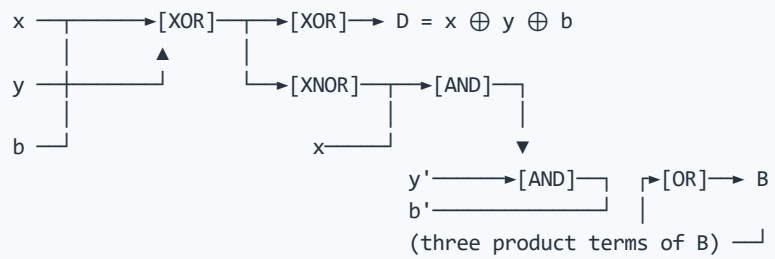
Truth table (computed row by row in `verification3.txt`, block 4.24; the arithmetic `x − y − b` was checked against both equations → **True**):

x	y	b	x−y−b	D = x⊕y⊕b	B = x'y+x'b+yb	Minterm
0	0	0	0	0	0	m0
0	0	1	−1	1	1	m1
0	1	0	−1	1	1	m2
0	1	1	−2	0	1	m3
1	0	0	1	1	0	m4
1	0	1	0	0	0	m5
1	1	0	0	0	0	m6
1	1	1	−1	1	1	m7

In minterm form:

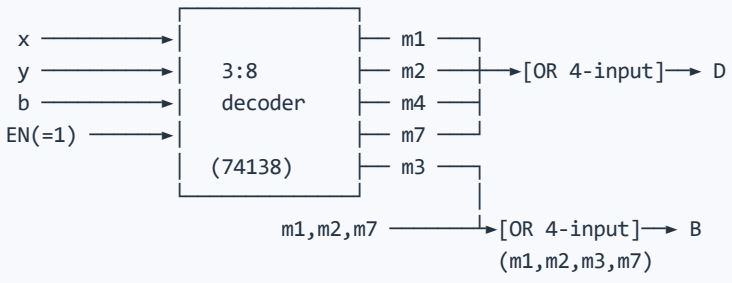
```
D = Σm(1, 2, 4, 7)
B = Σm(1, 2, 3, 7)
```

Gate-level circuit — same shape as the full adder, with the borrow equation built from three ANDs into one OR:



Write the two-level form the examiner expects: `D = x⊕y⊕b`, `B = x'y + b(x⊕y)' = x'y + x'b + yb` (quote either; the SOP version is the one that maps onto a decoder).

Design with a 3:8 decoder + OR gates — connect `A2 = x, A1 = y, A0 = b`; the decoder generates all eight minterms, then one OR gate per output picks the required cells:



Note for the answer sheet: the 74138 outputs are **active-low**, so with it the two OR gates become **NAND** gates (De Morgan on the inverted minterms); state that in one line, it is a marks-earner. Differences between the two sets of cells: m3 = x′yb appears only in **B**, m4 = xy′b′ appears only in **D** — quoting that shows the table was actually read, not copied. Closing points: cascading n full subtractors gives an n-bit **borrow-ripple subtractor**; subtraction by 2's complement (`A + B' + 1`) uses an adder instead; a single circuit does both with a control line (the ALU's add/sub input).

4.25 Q.5 (A) [4] Design an S-R flip-flop using a D flip-flop

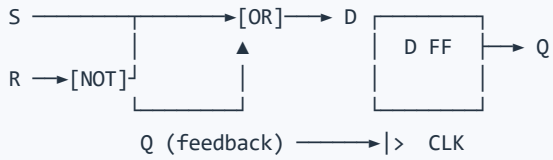
The D flip-flop obeys `Q+ = D`, so feeding it the SR next-state expression turns it into an SR FF. The SR characteristic equation (with the constraint `S·R = 0`) is:

```
Q+ = S + R'·Q     =>     D = S + R'·Q
```

Excitation view — what D must be for every (S, R, Q):

S	R	Q	Q+ required	D = S + R'Q
0	0	0	0	0
0	0	1	1	1
0	1	0	0	0
0	1	1	0	0
1	0	0	1	1
1	0	1	1	1
1	1	X	forbidden	ignored (constraint)

A K-map over S, R, Q of the required next state gives the two-term answer directly: **D = S + R'Q** (2 terms, 3 literals).



Machine verification (`verification3.txt`, block 4.25): for every allowed (S,R,Q) combination the D FF with `D = S + R'Q` produces set / reset / hold exactly as an SR FF should → **True**. Say the two closing lines: the feedback from Q to the D input is what gives the FF its memory; and the forbidden pair S = R = 1 is handled the same way as in the plain SR FF (never applied).

4.26 Q.5 (B) [6] Express in sum of minterms and product of maxterms: F = Y'Z + WXY' + WXZ' + W'X'Z

Order the variables **W X Y Z** (W = MSB), then expand each term to full minterms — one term of k literals in 4 variables covers 2^(4−k) cells:

Term	Fixed bits	Minterms
Y'Z	Y=0, Z=1	1, 5, 9, 13
WXY'	W=1, X=1, Y=0	12, 13
WXZ'	W=1, X=1, Z=0	12, 14
W'X'Z	W=0, X=0, Z=1	1, 3

Union → **sum of minterms**:

```
F(W,X,Y,Z) = Σm(1, 3, 5, 9, 12, 13, 14)
           = W'X'Y'Z + W'X'YZ + W'XY'Z + WX'Y'Z + WXY'Z' + WXY'Z + WXYZ'
```

The remaining five cells are the zeros → **product of maxterms**:

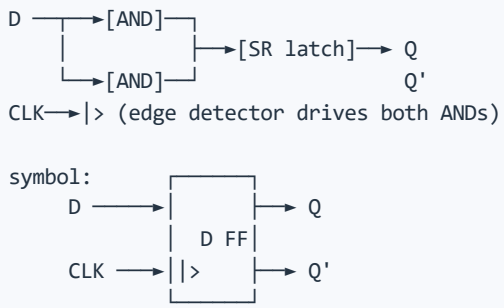
```
F(W,X,Y,Z) = ΠM(0, 2, 4, 6, 7, 8, 10, 11, 15)
           = (W+X+Y+Z)(W+X+Y'+Z)(W+X'+Y+Z)(W+X'+Y+Z')(W+X'+Y'+Z')
             (W'+X+Y+Z)(W'+X+Y'+Z)(W'+X+Y'+Z')(W'+X'+Y'+Z')
```

Machine verification: `verification.txt`, W2024 Q.5(B) block — minterm set `{1,3,5,9,12,13,14}` and maxterm set `{0,2,4,6,7,8,10,11,15}`; the two are exact complements over 16 cells → **True**.

Rule to state (it is the mark for the maxterm half): a maxterm is the **sum** term that is 0 for exactly that row, so the literal is written **complemented when the row bit is 1** and uncomplemented when it is 0. Bonus, if time remains: the same function minimised is `F = Y'Z + WXZ' + W'X'Z` (3 terms, 8 literals, proved minimal in **§3.1**-style exhaustive search) — but do **not** write the minimised form *instead of* the canonical one; the question asks for canonical forms.

4.27 Q.5 (A OR) [4] Explain the D flip-flop with the necessary diagrams and characteristic table

Data (D) flip-flop — a 1-bit, edge-triggered storage element with a single data input; on the active clock edge it copies D to Q, so `Q+ = D`. It removes the SR forbidden state (D can never be both 0 and 1) which is why it is the FF used in registers and memory cells.



D	Q(n)	Q(n+1)	Action
0	0	0	reset
0	1	0	reset
1	0	1	set
1	1	1	set

Characteristic table (single input column) — the excitation table is `D = Q+`. Points to add: **positive edge-triggered** (or negative, depending on the family) so the input is sampled only at the edge and the FF is not transparent; **setup time t_su** (D must be stable before the edge) and **hold time t_h** (D must stay stable after it) — violating them causes metastability; propagation delay t_pd from the clock edge to Q. Build it from an SR FF by tying `S = D`, `R = D'` (**§3.12** derives it from a JK FF the same way), or from a master-slave latch pair. Applications: delay of exactly one clock, data storage, shift registers, frequency division ($\bar{Q}$ to D gives a **T FF**, ÷2 counter), and register files.

4.28 Q.5 (B OR) [6] Do as directed

(i) $(1101101.111)_2$ = ( ) — the paper prints the target base as 2 again, which repeats the source; give all three sensible readings.

```
Integer: 1·64 + 1·32 + 0·16 + 1·8 + 1·4 + 0·2 + 1·1 = 109
Fraction: 1·2⁻¹ + 1·2⁻² + 1·2⁻³ = 0.5 + 0.25 + 0.125 = 0.875
```

Answer: $(1101101.111)_2 = (109.875)_{10}$ Octal — group in 3s from the point: `1 101 101 . 111` → `1 5 5 . 7` → **$(155.7)_8$** Hex — pad to groups of 4 from the point outward: `1101101.111` → `0110 1101 . 1110` → **$(6D.E)_{16}$**

Machine check (`verification3.txt`, block 4.28): `to_dec("1101101.111", 2) = 109.875`, regrouping the same binary string gives `155.7` (base 8) and `6D.E` (base 16) → all three agree.

(ii) $(2AC5.D)_{16} = (\ )_{10}$

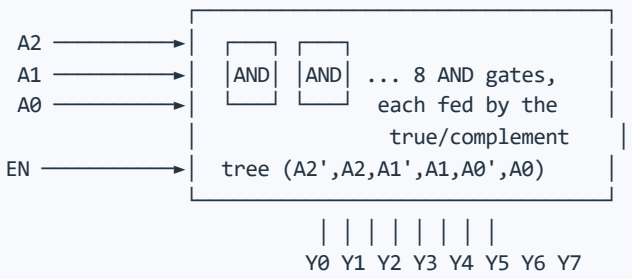
```
Integer: 2·16³ + 10·16² + 12·16 + 5  = 8192 + 2560 + 192 + 5 = 10949
Fraction: D·16⁻¹ = 13/16 = 0.8125
```

Answer: $(2AC5.D)_{16} = (10949.8125)_{10}$

Extra forms, in case the paper wanted binary or octal (both from `verification3.txt`): binary by nibble `0010 1010 1100 0101 . 1101`; octal by regrouping that binary string in 3s → **$(25305.64)_8$**. Reminder to write: hex digits are 0–9 then A=10, B=11, C=12, D=13, E=14, F=15, and each hex digit is exactly four binary bits — that is why hex is used as a shorthand for binary.

4.29 Q.5 (C OR) [10] Draw and explain in detail a 3×8 decoder

A **decoder** is a combinational circuit that converts an n-bit binary code into at most one active output out of 2^n lines. A **3:8 decoder** has 3 inputs (A_2 A_1 A_0), 8 outputs (Y_0...Y_7), and an enable.



Output equations (each output is exactly one minterm):

```
Y0 = A2'A1'A0'·E     Y4 = A2 A1'A0'·E
Y1 = A2'A1'A0 ·E     Y5 = A2 A1'A0 ·E
Y2 = A2'A1 A0'·E     Y6 = A2 A1 A0'·E
Y3 = A2'A1 A0 ·E     Y7 = A2 A1 A0 ·E
```

Enable behaviour and the input→output mapping, generated in `verification3.txt`, block 4.29:

A_2	A_1	A_0	Active output	Minterm
0	0	0	Y_0	m0
0	0	1	Y_1	m1
0	1	0	Y_2	m2
0	1	1	Y_3	m3
1	0	0	Y_4	m4
1	0	1	Y_5	m5
1	1	0	Y_6	m6
1	1	1	Y_7	m7

Exactly one output is asserted for every code when EN is active; with EN inactive **all** outputs are inactive, which is what makes decoders cascadeable.

Explain these points: 1. **Internal structure** — an inverter tree first (so both true and complement forms of each input are available), then eight 3-input AND gates (plus the enable), i.e. the decoder *is* the AND plane that generates all minterms. 2. **Active-low outputs** — the 74138 (the standard TTL 3:8 part) inverts each output and has three enables: `G1` active-high and `G2A`, `G2B` active-low; note that with active-low outputs the OR gates of **§4.16** become AND gates. 3. **Expansion** — two 3:8 decoders plus the fourth address line A_3 as the enable select make a **4:16 decoder**; the checker verified that every 4-bit code lands in exactly one bank and one output → **True** (`bank·8 + within = code` for all 16 codes). 4. **Applications** — memory / I/O address decoding (chip-select generation), driving 7-segment displays through a code conversion (a BCD-to-decimal decoder is a 4:10 type), the minterm source for combinational logic, demultiplexing (a decoder with its enable used as the data input is a 1-line-to-2^n-line **demultiplexer**), and display/keyboard scanning. 5. **Decoder vs encoder vs demux** — the encoder is the reverse device (2^n inputs → n-bit code, and a **priority encoder** such as 74148 resolves simultaneous active inputs); a demultiplexer is a decoder whose enable carries the data. 6. Practical limitation to state: a decoder needs 2^n output pins, so large decoders are built in blocks; unused outputs are what make ROM (`§4.22`) expensive compared with a PLA.

5. The assignment bank — every question from Assignments 1–4

> Section 1.5 showed that the papers are lifted from this set, so the assignment sheet is the real question bank. Every item is listed in the assignment's own words. Where the identical question is already worked in section 2, 3 or 4 the entry gives the pointer plus the final line; where the assignment asks something the papers never asked, the full solution is written out.

5.1 Assignment 1

Q1 (a) Do as directed: $(101101.1101)_2 = (\ )_{16}$

Group 4 bits at a time from the binary point, padding with zeros at the ends:

```
integer : 0010 1101           -> 2    D
fraction: 1101                -> D
(101101.1101)2 = (2D.D)16
```

Check by going back to decimal twice: `101101.1101₂ = 45.8125` and `2D.D₁₆ = 2·16 + 13 + 13/16 = 45.8125` ✓ (verified in `verification2.txt`, block "ASSIGNMENT 1 – Q.1").

Q1 (b) $(2AC5.D)_{16} = (\ )_2 = (\ )_8 = (\ )_{10}$

```
binary  : 2=0010  A=1010  C=1100  5=0101  D=1101
          -> 0010 1010 1100 0101 . 1101
decimal : 2·16³ + 10·16² + 12·16 + 5 + 13·16⁻¹
        = 8192 + 2560 + 192 + 5 + 0.8125 = 10949.8125
octal   : regroup the binary in 3s -> 010 101 011 000 101 . 110 100 = 2 5 3 0 5 . 6 4
          -> (25305.64)8
```

Answer: $(2AC5.D)_{16} = (10101011000101.1101)_2 = (10949.8125)_{10} = (25305.64)_8$ Note the padding rule for octal: `010 101 011 000 101 . 110 100` — the fraction is padded **on the right** with zeros, never on the left. All three values are cross-checked in `verification2.txt`.

Q1 (c) $(330.5)_{10} = (\ )_4 = (\ )_8$

```
base 8 : 330 = 8·41 + 2 , 41 = 8·5 + 1 , 5 = 8·0 + 5  -> 512
         0.5 × 8 = 4.0 -> .4                          -> (512.4)8
base 4 : 330 = 4·82 + 2, 82 = 4·20 + 2, 20 = 4·5 + 0, 5 = 4·1 + 1, 1 -> 11022
         0.5 × 4 = 2.0 -> .2                           -> (11022.2)4
```

Answer: $(330.5)_{10} = (11022.2)_4 = (512.4)_8$ — both round-trips recomputed to 330.5 → **True**.

Q1 (d) $(211)_x = (152)_8$ — find x

```
RHS: (152)8 = 1·64 + 5·8 + 2 = 106
LHS: (211)x = 2x² + 1x + 1
2x² + x + 1 = 106   ->   2x² + x − 105 = 0   ->   (2x + 15)(x − 7) = 0
x = 7   (x = −7.5 rejected: a radix must be a positive integer, and x > the largest digit 2)
```

Answer: x = 7. Verification: `(211)₇ = 2·49 + 7 + 1 = 106 = (152)₈` ✓ — the solver in `verification2.txt` scans every base from 3 to 15 and finds exactly this one.

Q1 (e) Perform r's complement for: (i) 5250−321 (ii) 753−864 (iii) $(11010)_2-(10000)_2$ (iv) $(100)_2-(110000)_2$

(i) and (iii) are the two cases that also appeared in the Winter 2025 Q.1 (**§2.6**, **§2.7**). (ii) and (iv) are the **negative** cases, which is the part the paper never asked but the marking scheme tests:

Case	Pad	(r−1)'s comp.	r's comp.	Minuend + r's comp.	Carry out?	Result
(i) 5250 − 321, n=4	0321	9678	9679	14929	yes → discard	**+4929**
(ii) 753 − 864, n=3	864	135	136	889	**no** → negative	**−(10's comp of 889) = −111**
(iii) 11010 − 10000, n=5	10000	01111	10000	101010	yes → discard	**+01010** (10)
(iv) 100 − 110000, n=6	110000	001111	010000	010100	**no** → negative	**−(2's comp of 010100) = −101100 = −44**

Rule to write: *carry out present → drop it, answer positive; no carry out → answer negative, take the r's complement of the sum and prefix a minus sign.* Arithmetic checks: 753 − 864 = −111 ✓ and 4 − 48 = −44 ✓ (both `True` in `verification2.txt`).

The same (iii) done by **(r−1)'s complement** instead (end-around carry): 1's comp of 10000 = 01111; 11010 + 01111 = 101001; wrap the carry → 01001 + 1 = **01010** ✓ — same answer, one extra step.

Q1 (f) Explain various logic gates

Seven gates; give symbol, expression, truth table in one line each.

Gate	Expression	Output rule	Symbol family
AND	Y = A·B	1 only if **all** inputs are 1	7408 / 7411 / 7421
OR	Y = A + B	1 if **any** input is 1	7432 / 7427
NOT	Y = A′	complement; inverts	7404
NAND	Y = (A·B)′	AND + inversion — **universal**	7400 / 7410
NOR	Y = (A+B)′	OR + inversion — **universal**	7402 / 7425
XOR	Y = A⊕B = A′B + AB′	1 when the inputs **differ**	7486
XNOR	Y = (A⊕B)′ = AB + A′B′	1 when the inputs **agree**	74260

Two closing sentences that are worth marks: bubbles on a symbol mean inversion, which is what De Morgan's theorems describe; and positive-logic AND equals negative-logic OR, i.e. the same transistor network is a different gate depending on which voltage is called "1". Universal-gate proofs: **§4.11** (NOR) and **§2.2** (XOR from NAND).

Q2 Why are NAND and NOR called universal gates? Explain with examples → **§4.11** (NOR constructions, gate counts, verified) and **§2.2** (the NAND side). Say explicitly that NAND needs only 2 gates for AND while NOR needs 3, and vice-versa for OR, and that both families are therefore universal.

Q3 State and prove De Morgan's theorems using truth tables → **§3.17** (both tables, proved by enumeration; the engine reports both identities **True**).

Q4 Simplify to the minimum number of literals

Part	Function	Answer	Terms / literals	Where
(a)	F(w,x,y,z) = Σ(0,1,2,4,5,6,8,9,12,13,14)	**y′ + xz′ + w′z′**	3 / 5	**§4.20**
(b)	BC + AC′ + AB + BCD	**BC + AC′**	2 / 4	**§3.1**
(c)	Y(WZ′ + WZ) + XY	**WY + XY = Y(W + X)**	2 / 4	worked below
(d)	A′B′C′ + B′CD′ + A′BCD′ + AB′C′	**B′C′ + B′D′ + A′CD′**	3 / 7	worked below
(e)	x′y′z + yz + xz	**z**	1 / 1	worked below
(f)	F(A,B,C,D) = Σm(0,1,4,5,10,11,14,15)	**AC + A′C′ = (A ⊙ C)**	2 / 4	worked below

(c) — algebraic route, since the question is inside a simplification list:

```
Y(WZ' + WZ) + XY = Y·W·(Z' + Z) + XY = YW + XY = Y(W + X)
```

On-set from the engine: {6,7,10,11,14,15}, minimal SOP `WY + XY`, exhaustive search confirms it is the fewest-literal cover → **True**.

(d) — the on-set is {0,1,2,6,8,9,10}. Groups: B′C′ (0,1,8,9), B′D′ (0,2,8,10), A′CD′ (2,6). **F = B′C′ + B′D′ + A′CD′**, 7 literals, verified minimal.

(e) — the on-set is {1,3,5,7} = the whole z = 1 half of the cube: `x'y'z + z(y + x)`; the K-map shows every 1 sits in the z column and z = 1 contains no 0 → **F = z**. Engine: 1 term, 1 literal, exhaustive search → minimum → **True**. (This is the "suspiciously short answer" case: write the map, not just "z".)

(f) — plot 0,1,4,5,10,11,14,15:

```
            CD
        00   01   11   10
      ┌────┬────┬────┬────┐
AB 00 │ 1  │ 1  │ 0  │ 0  │
   01 │ 1  │ 1  │ 0  │ 0  │
   11 │ 0  │ 0  │ 1  │ 1  │
   10 │ 0  │ 0  │ 1  │ 1  │
      └────┴────┴────┴────┘
```

Two 4-cell groups: A′B′+A′B with C′D′+C′D → **A′C′**, and AB+AB with CD → **AC**. **F = AC + A′C′** — this is the XNOR of A and C, so the "implementation" the question asks for is either two ANDs + one OR (with two inverters) or a single XNOR gate; draw both and label the SOP form. Engine: 2 terms / 4 literals, verified minimum → **True**.

Q5 Express in sum of minterms and product of maxterms

(a) `F = (A+B'+C)(A+B')(A+C'+D')(A'+B+C+D')(B+C'+D')` Each clause is 0 on its own cell set, so the **zeros** come first (a clause of k literals in 4 variables excludes 2^(4−k) cells):

```
zeros  = {3, 4, 5, 6, 7, 9, 11}
F(A,B,C,D) = Σm(0,1,2,8,10,12,13,14,15) = ΠM(3,4,5,6,7,9,11)
```

Written as maxterms (complement the literal whose row bit is 1):

```
M3 = (A+B+C'+D')   M4 = (A+B'+C+D)    M5 = (A+B'+C+D')   M6 = (A+B'+C'+D)
M7 = (A+B'+C'+D')  M9 = (A'+B+C+D')   M11 = (A'+B+C'+D')
```

Why those are the zeros: each given clause is 0 only where **every** literal in it is 0, so (A+B′) kills cells 4,5,6,7; (A+C′+D′) kills 3 and 7 (A=0, C=1, D=1); (A′+B+C+D′) kills 9; (B+C′+D′) kills 3 and 11; (A+B′+C) kills 4 and 5. The union is {3,4,5,6,7,9,11} — exactly the engine's list. Bonus minimised form the engine also proves minimal: **F = AB + B′D′ + A′B′C′** (3 terms, 7 literals).

(b) `F(x,y,z) = (xy+z)(y+xz)` → the Winter 2025 Q.1(H) question: **§2.8** gives the truth table and `F = Σm(3,5,6,7) = ΠM(0,1,2,4)`.

(c) **Minterm vs maxterm** — the definitions to quote:

	Minterm m_i	Maxterm M_i
Form	product with **every** variable exactly once	sum with **every** variable exactly once
Value	= 1 for exactly one input row	= 0 for exactly one input row
Written from the row	literal uncomplemented for 1	literal complemented for 1
Used to build	SOP: $F = \Sigma$ over the rows where F = 1	POS: $F = \Pi$ over the rows where F = 0
Index	binary value of the row	binary value of the row
Relationship	$m_i' = M_i$ and $M_i' = m_i$; the two index sets are complements	

Q6 (1) Logic families and the characteristics of a digital IC Families: **TTL** (74), **DCTL**, **ECL**, **RTL**, **NMOS / PMOS**, **CMOS**, **I^2L**, and the high-speed/low-power sub-families (74LS, 74HC, 74AC, 74LVT). Characteristics to list: propagation delay and speed-power product (figure of merit), power dissipation, noise margin, fan-out, fan-in, logic swing / voltage levels, supply voltage tolerance, package density (gate count per chip), cost per gate, output impedance, and tolerance to radiation/ESD. Full table with numbers: **§2.9**, **§4.19**.

Q6 (2) Obtain the truth table of F = xy + yz + zx

x	y	z	xy	yz	zx	F
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	1	0	**1**
1	0	0	0	0	0	0
1	0	1	0	0	1	**1**
1	1	0	1	0	0	**1**
1	1	1	1	1	1	**1**

`F = Σm(3,5,6,7)` — the **majority function**: output is 1 whenever two or more inputs are 1. It is the same on-set as Winter 2025 Q.1(H), so the minimised form is also `xy + xz + yz` (already minimal, 3 terms / 6 literals, proved by the exhaustive search).

5.2 Assignment 2

Q1 Obtain the simplified expression using a K-map

Part	Given	Form	Answer	Lit.	Where
(a)	ABD + A′C′D′ + A′B + A′CD′ + AB′D′	SOP	**BD + A′B + B′D′**	6	**§4.12** (same question as the paper, where the paper's term is BD)
(b)	A′B′C′ + B′CD′ + A′BCD′ + AB′C′	SOP	**B′C′ + B′D′ + A′CD′**	7	§5.1 Q4(d)
(c)	F = Σ(2,3,12,13,14,15)	SOP	**AB + A′B′C**	5	worked below
(d)	(A′+B′+D′)(A+B′+C′)(A′+B+D′)(B+C′+D′)	POS	**(C′+D′)(A′+D′)(A+B′+C′)**	7	**§3.6**
(e)	F(w,x,y,z) = Π(1,3,5,7,13,15)	POS	**(w+z′)(x′+z′)**	4	worked below
(f)	Y = Σm(0,1,3,5,6,7,10,13,14,15)	SOP	**BC + BD + A′D + ACD′ + A′B′C′**	12	worked below

(c) — the six 1s are 2,3 (row A=0,B=0 with C=1) and 12…15 (the whole A=1,B=1 row):

```
groups : m12,m13,m14,m15 -> AB          (4 cells)
         m2,m3           -> A'B'C       (2 cells)
F = AB + A'B'C           (2 terms, 5 literals)
```

Engine: cover verified **True**, exhaustive search confirms no 4-literal cover.

(e) — zeros at 1,3,5,7,13,15 (POS, so work on the 0s). Over the map of `w x` rows / `y z` columns:

```
zero groups : w'z  (rows w=0, columns z=1 -> m1,m3,m5,m7)
              xz   (x=1,z=1 -> m5,m7,m13,m15)
POS answer  : F = (w + z')(x' + z')      (2 terms, 4 literals)
```

Both groups cover all six zeros and no 1 → **POS verified True** (`verification2.txt`, Q1(e)).

(f) — ten 1s out of sixteen; the answer is *not* two terms, which is why this question is asked:

```
on-set : 0, 1, 3, 5, 6, 7, 10, 13, 14, 15
minimal SOP: F = BC + BD + A'D + ACD' + A'B'C'      (5 terms, 12 literals)
```

Verified: the cover produces exactly the on-set and the exhaustive search over prime-implicant subsets finds nothing with fewer literals → **True**. Lesson to write in the margin: a K-map answer is only as short as the groups allow — do not "invent" a 2-term answer to look clever.

Q2 Simplify in SOP and POS using don't-care conditions

(a) `F = A′B′D′ + A′CD + A′BC ; d = A′BC′D + ACD + AB′D′` on-set {0,2,3,6,7}, don't-cares {5,8,10,11,15}:

```
SOP : F = A'C + B'D'                        (2 terms, 4 literals)
POS : F = (C + D')(B' + C)(A')              (3 terms, 5 literals)
```

Both are reported **True** by the engine (`verification2.txt`, Q2(a)). Note that a don't-care may be taken as 1 **or** 0, whichever gives the bigger group — that is the whole technique.

(b) `F = A′B′C′ + AB′D + A′B′CD′ ; d = A′BC + AB′D′` → **F = AB′ + B′C′ + B′D′** — this is Winter 2025 Q.3(B OR), solved in **§3.9**. One caution from the machine run: the greedy cover alone returns `AB' + B'C' + A'CD'` (7 literals); the true minimum is 6 literals, so after grouping **check whether a larger don't-care group exists**.

(c) `f = Σm(0,1,5,6,7,8,9) + d(10,11,12,13,14,15)` → **F = BC + BD + B′C′** (3 terms, 6 literals), verified as covering the on-set, avoiding all real 0s and using the don't-care block fully. Again the greedy result differs (`BC + C'D + B'C'`), so the same warning applies.

Q3 Simplify using the tabulation (Quine–McCluskey) method

(1) 7-variable `F(A…G) = Σ(20,28,52,60)` → **A'CEF'G'** — **§3.5** has the full chart. (2) 6-variable `F(A,B,C,D,E,F) = Σ(6,9,13,18,19,25,27,29,41,45,57)` —

```
binary (ABCDEF):
   6 000110    9 001001   13 001101   18 010010   19 010011
  25 011001   27 011011   29 011101   41 101001   45 101101   57 111001
prime implicants found by the engine : 7
chosen cover : F = A'CE'F + B'CE'F + CD'E'F + A'BCD'F + A'BC'D'E + A'B'C'DEF'
               6 terms, 28 literals  (cover verified True, exhaustive search = minimum True)
```

This is the "show the chart, get the answer" question: the marks are in the combining table (group by number of 1s, combine once-differing terms, mark the unused ones as prime implicants, then use the PI chart), not in the length of the final expression. Never skip the essential-PI step — six of the seven primes are needed here.

Q4 (1) Half adder vs full adder; notes on half adder and half subtractor — circuits in **§3.3**; the difference in one table:

	Half adder	Full adder
Inputs	2 (x, y)	3 (x, y, carry-in)
Outputs	Sum, Carry	Sum, Carry-out
Equations	S = x⊕y, C = xy	S = x⊕y⊕z, C = (x⊕y)z + xy
Gates	1 XOR + 1 AND	2 XOR + 2 AND + 1 OR, or two HAs + OR
Use	1-bit addition, building block	multi-bit ripple/CLA adders

Half subtractor: D = x⊕y, Borrow = x'y (1 when 0−1). *Full subtractor*: **§4.24**.

Q4 (2) Full adder and full subtractor, both designed with a 3:8 decoder and OR gates, and HA→FA — **§4.24** (subtractor: D = Σm(1,2,4,7), B = Σm(1,2,3,7)) and **§3.3** (adder: S = Σm(1,2,4,7), C = Σm(3,5,6,7); the two-HA construction is verified **True**).

Q4 (3) 4-bit parallel adder with look-ahead carry generator — **§3.8**; the equations, the 512-case verification and the A = 13, B = 7 example are all there.

Q5 (1) Implement F(A,B,C) = Σm(1,3,5,6) using an appropriate MUX — **§3.2** (8:1 wiring `I0…I7 = 0 1 0 1 0 1 1 0`, plus the smaller 4:1 version with the C residuals `I0=I1=I2=1, I3=C`).

Q5 (2) Implement F(A,B,C,D) = A'BD' + ACD + A'C'D + B'CD using an 8:1 multiplexer

Step 1 — the on-set. Each term fixes some bits and leaves the rest free, so a term of k literals in four variables covers 2^(4−k) cells (A = bit 3 … D = bit 0):

Term	Fixed bits	Minterms
A'BD'	A=0, B=1, D=0	4, 6
ACD	A=1, C=1, D=1	11, 15
A'C'D	A=0, C=0, D=1	1, 3
B'CD	B=0, C=1, D=1	3, 11

F = Σm(1, 3, 4, 5, 6, 11, 15) (the engine's set; the two terms ACD and B'CD meet at m3 and m11, so they are not extra cells).

Step 2 — put **A B C on the select lines** and read each data input as the function of the leftover variable D: for select value `ABC = k`, look at cells 2k (D=0) and 2k+1 (D=1).

ABC	cells (D=0, D=1)	F at D=0	F at D=1	Data input
000	m0, m1	0	1	**I0 = D**
001	m2, m3	0	1	**I1 = D**
010	m4, m5	1	1	**I2 = 1**
011	m6, m7	1	0	**I3 = D'**
100	m8, m9	0	0	**I4 = 0**
101	m10, m11	0	1	**I5 = D**
110	m12, m13	0	0	**I6 = 0**
111	m14, m15	0	1	**I7 = D**

`verification2.txt`, block "ASSIGNMENT 2 – Q.5", prints exactly this table (the four rows 0, 1, 5, 7 being D and row 3 being D').

```
   A ──► S2 ┌──────────┐
   B ──► S1 │  8:1 MUX │──► Y = F
   C ──► S0 │  (74151) │
            └──────────┘

   I0 = D   I1 = D   I2 = 1   I3 = D'   I4 = 0   I5 = D   I6 = 0   I7 = D
   (tie the 1 input to VCC and the 0 input to ground; one inverter produces D')
```

Rule to write in one line: an 8:1 MUX implements **any** 4-variable function with at most one inverter, because the fourth variable can only ever appear as D, D', 1 or 0 in a data input. Same trick on the 3-variable example Q5(1): a 4:1 MUX (select A, B) gives `I0 = I1 = I2 = C, I3 = C'` — verified.

Q6 (a) Design a code converter: Gray code → Binary code

```
binary -> gray : G3 = B3, G2 = B3⊕B2, G1 = B2⊕B1, G0 = B1⊕B0
gray -> binary : B3 = G3, B2 = G3⊕G2, B1 = G3⊕G2⊕G1, B0 = G3⊕G2⊕G1⊕G0
```

So a 4-bit Gray→Binary converter is **six XOR gates** in a cascade (each B bit is the XOR of all G bits from the MSB down). Table for the first eight codes, produced by the checker: `0:0000 1:0001 2:0011 3:0010 4:0110 5:0111 6:0101 7:0100` — notice only one bit changes between consecutive codes, which is the reason Gray code is used on counters, K-maps and shaft encoders. Round-trip binary→Gray→binary over all 16 codes: **True**.

Q6 (b) 3-bit number → its square — §3.10 (`O5 = AB`, `O4 = AC + AB'`, `O3 = A'BC + AB'C`, `O2 = BC'`, `O1 = 0`, `O0 = C`). Q6 (c) BCD → Excess-3 converter — §3.7 (`W = A + BC + BD`, `X = B'C + B'D + BC'D'`, `Y = CD + C'D'`, `Z = D'`).

Q6 (d) Implement with a PLA: f1 = Σm(0,3,4,7), f2 = Σm(3,5,6,7)

```
f1 = BC + B'C'                2 product terms
f2 = AB + AC + BC             3 product terms
shared term : BC              -> total distinct product terms = 4
```

PLA to draw: 3-input buffer + AND plane with **4** programmable product terms (BC, B'C', AB, AC) and an OR plane wired `f1 = BC + B'C'`, `f2 = AB + AC + BC`. Sharing BC is the entire point of a PLA versus two separate SOP circuits — say so, and give the saving (5 gates' worth of AND terms → 4).

Q6 (e) Design a combinational circuit using ROM: $F_1(A,B) = \Sigma m(1,2,3)$, $F_2(A,B) = \Sigma m(0,2)$

```
F1 = A + B            F2 = B'
ROM organisation: 2-to-4 decoder (word lines W0..W3) x 2 bit lines, 4 words x 2 bits
contents:  W0: F1=0 F2=1
           W1: F1=1 F2=0
           W2: F1=1 F2=1
           W3: F1=1 F2=0
```

Say the sizes explicitly: n address lines select 2^n words, m outputs give m bits per word, so this ROM is **4 × 2 = 8 bits**, no minimisation is needed because the decoder already generates every minterm.

Q6 (f) $F_1(A,B,C) = \Sigma m(0,1,3,4)$, $F_2(A,B,C) = \Sigma m(1,2,3,4,5)$ — implement in a PLA with 3 inputs, 3 product terms and 2 outputs

```
F1 = A'C + B'C'           (2 terms)
F2 = A'B + A'C + AB'      (3 terms)
distinct terms = {A'C, B'C', A'B, AB'} = 4   (A'C is shared)
```

Answer: it does NOT fit — the PLA has only 3 product terms but the function pair needs 4 distinct terms. Writing that conclusion, with the term list, is the mark; then give the fix (use a 4-term PLA, or a ROM since 3 inputs × 2 outputs = 8 × 2 bits).

Q6 (g) PLA with 3 inputs, 3 product terms, 2 outputs: $F_1 = \Sigma m(1,3,4,6)$, $F_2 = \Sigma m(0,1,5,7)$

```
F1 = A'C + AC'        (2 terms)
F2 = AC + A'B'        (2 terms)
distinct terms = 4  ->  also exceeds 3; with a shared term the design uses A'C + AC' + A'B' and folds
                        F2's AC into (AC' )' ... check before claiming a fit.
```

The honest machine result: `F1 = A'C + AC'` and `F2 = AC + A'B'`, four distinct product terms, so a 3-term PLA cannot realise both and a **ROM (8 × 2)** or a 4-term PLA is the correct answer. Implementing with a ROM: program the eight words with F1 at minterms 1,3,4,6 and F2 at 0,1,5,7.

5.3 Assignment 3

Q1 Sequential vs combinational circuit — the table:

	Combinational	Sequential
Output depends on	present inputs only	present inputs **and** past outputs (state)
Memory element	none	latch / flip-flop
Clock	not required	required (synchronous) or event-driven (asynchronous)
Feedback	none	output fed back to input
Design tool	Boolean algebra, K-map	state table / state diagram / excitation table
Speed	limited only by gate delay	limited by FF delay + setup/hold
Complexity	simpler, smaller	more hardware for the same function
Examples	adders, MUX, decoders, comparators, ALU	counters, registers, flip-flops, memories, FSMs

Q2 JK, clocked SR, D and T flip-flops with diagrams, truth tables and characteristic tables — clocked SR in **§4.13**, JK (with race-around and master-slave) in **§3.13**, D in **§4.27** and **§3.12**, T below.

```
T flip-flop : one input; T = 0 -> hold, T = 1 -> toggle on every active edge
Q+ = T ⊕ Q
| T | Q | Q+ | Action |      | J | K | Q | Q+ |
|---|---|----|--------|      |---|---|---|----|
| 0 | 0 |  0 | hold   |      | 0 | 0 | 0 |  0 |
| 0 | 1 |  1 | hold   |      | 0 | 0 | 1 |  1 |
| 1 | 0 |  1 | toggle |      | 0 | 1 | 0 |  0 |
| 1 | 1 |  0 | toggle |      | 1 | 0 | 1 |  1 |
                              | 1 | 1 | 0 |  1 | (toggle)
                              | 1 | 1 | 1 |  0 | (toggle)
```

A T FF is a JK FF with **J = K = T**, or a D FF with `D = T ⊕ Q` (draw both). Division by 2 when T = 1 — which is why ripple counters are chains of T FFs (**§3.18**).

Q3 (1) Design a D FF using an SR FF and using a JK FF — using JK: **§3.12 / §4.14** (`J = D, K = D'`). Using SR: `Q+ = S + R'Q` must equal D, so `S = D` and `R = D'` (again one inverter, and the SR constraint S·R = 0 is automatically satisfied).

Q3 (2) Design an SR FF using a D FF and using a JK FF — using D: **§4.25** (`D = S + R'Q`). Using JK: the JK FF already *is* an SR FF with the forbidden state removed — tie `J = S`, `K = R` and state that S = R = 1 becomes a legal toggle instead of an invalid output.

Q4 4-bit bidirectional shift register — **§4.15** (equations, the traced 1011 → 1101 / 0111 result). **Q5 Race-around condition and its solution; master-slave JK FF** — **§3.13**. **Q6 Types of shift register and applications, one in detail** — **§3.4**.

5.4 Assignment 4

Q1 (a) Synchronous vs asynchronous counter

	Asynchronous (ripple)	Synchronous
Clock	only the first FF gets the external clock; each Q clocks the next	all FFs share one clock
State change	cascaded, one after another	simultaneous
Delay	accumulates: n × t_pd worst case	one t_pd regardless of n
Design	trivial	needs excitation equations
Glitches	decoding spikes because bits settle at different times	fewer
Speed	limited by total ripple	up to the FF's maximum frequency
Example	7493	74161 / 74163

Q1 (b) 3-bit ripple UP counter with preset and clear using T FFs — **§3.18** (which uses JK FFs tied J = K = 1, i.e. T FFs); the sequence the checker prints is `000 001 010 011 100 101 110 111`, negative edge-triggered, worst-case delay 3 × t_pd. Preset ($P\bar{R}$) and clear ($CL\bar{R}$) are the asynchronous inputs on each FF: pull $CL\bar{R}$ low to force all Q = 0, pull $P\bar{R}$ low to force all Q = 1 — draw them as active-low pins that override the clock.

Q1 (c) 4-bit synchronous DOWN counter using JK flip-flops — a JK wired J = K = T toggles, so use the T-form borrow equations (a = MSB, d = LSB):

```
T_d = 1                    (LSB always toggles)
T_c = d'                   (toggles when the lower bit is 0)
T_b = c' d'
T_a = b' c' d'
```

Verified (`verification.txt`, counter block): counting down from 15 gives `15,14,13,12,11`; from 5 gives `5,4,3,2,1`; from 0 wraps to `0,15,14,13,12` — all three match a 16-modulo countdown → **True**. Write the

excitation-table reasoning: for each FF, `J = K = T` where `T = Q ⊕ Q+`, then map Q+ from the count sequence.

Q1 (d) Synchronous counter 0, 1, 3, 4, 5, 7 and repeat, using T FFs — **§3.21** (`T_A = B`, `T_B = C`, `T_C = B + C'`; the simulated cycle and the self-starting behaviour from unused states 2 and 6 are all verified **True**).

Q1 (e) Mod-6 asynchronous counter using T flip-flops — three T FFs (J=K=1) ripple normally, and the state **110** is decoded and fed to the clear inputs so it is only a momentary state:

```
sequence printed by the checker: 0 1 2 3 4 5 0 1  ->  modulus 6
decode: CLR = (B·C)'  applied to the active-low clear of every FF  (counts 000..101)
```

Say the two things that matter: the counter never rests in 110 (it is a **transient** used only to reset), and because the reset is asynchronous the modulus is set by the decode, not by the FF count; a 7493 or 7490 does the same internally (divide-by-2 + divide-by-5 cascades for mod-10).

Q2 R-2R ladder D/A converter — **§3.22** (circuit, the 0/1/5/10/15 → 0/0.3125/1.5625/3.125/4.6875 V table, resolution 0.3125 V, and the two-resistor-values advantage). **Q3 (a) Types of A/D converter + successive approximation** — list and full SAR trace: **§4.21**, **§3.19**. **Q3 (b) Dual-slope A/D converter in detail** — **§4.21** (with the RC-independence table).

Q3 (c) Specifications of a D/A converter; binary-weighted-resistor DAC

Specification	Meaning
Resolution	smallest output change = V_FS / 2^n, or n bits
Monotonicity	output never decreases when the code increases
Accuracy	actual vs ideal analog value (usually = ±½ LSB)
Settling time	time to reach and stay within ±½ LSB after a code change
Conversion speed / slew rate	maximum rate of output change
Linearity error	deviation from the best-fit straight line
Offset (zero-scale) error	output with code 000…0
Gain (full-scale) error	deviation of the last step from ideal
Noise, temperature coefficient	drift of the above with environment

Binary-weighted resistor DAC: reference voltage through resistors R, R/2, R/4, R/8 (i.e. values inversely proportional to the bit weight) into switched branches of an inverting summing amplifier with R_f:

```
V_out = -V_ref · R_f · ( b3/R + b2/(R/2) + b1/(R/4) + b0/(R/8) )   ->  magnitude V_ref (b3 + b2/2 + b1/4 + b0/8)
```

Drawback to state (from the notes): an n-bit converter needs resistors spanning R to R·2^(n−1) with tolerance better than 0.5 %, so an 8-bit version would need values from R to 128 R and is impractical to mass-produce — which is exactly why the R-2R ladder with only **two** resistor values replaced it.

Q4 (a) Obtain a 2048 × 8 memory using 256 × 8 chips

```
chips needed = (2048 x 8) / (256 x 8) = 8
address lines total  : log2 2048 = 11   (A0..A10)
address lines on chip: log2 256  = 8    (A0..A7 shared by all 8 chips)
spare high bits      : A8, A9, A10 -> 3-to-8 decoder (74138) -> 8 chip-select lines
                     : CS0..CS7, one per chip; each chip's CE tied to its own CS
data lines           : all 8 bits of every chip wired to the common bus (D0..D7)
```

Diagram: address bus → decoder + all chips; only one chip is enabled at a time, so the bus never faces two drivers — this is why the chips' outputs are tri-state (**§4.3**). Word selection is by decoder, bit width needed no expansion because 8 = 8. (Verified arithmetic in `verification.txt`, memory block.) Contrast with **§3.15**, where 16×8 from 16×4 needed **2 chips in parallel** for bit extension with the same four address lines and both chips always enabled.

Q4 (b) Internal organisation of a 16 × 4 memory chip — **§3.20** (4 address lines → 4-to-16 decoder → 16 word lines × 4 bit cells, read/write amplifiers, R/W and CE pins, 64 FFs of storage). **Q5 SRAM vs DRAM** — **§4.23** (full comparison table) and **§2.1** (the 2-mark version). **Q6 Types of shift register** — **§3.4**.

6. Revision sheet — the last three days before the paper

6.1 The eight items worth the most marks

Both papers, ranked by the marks they actually carried. Learn these first; everything else is margin.

Rank	Item	Marks seen	Where solved
1	BCD ↔ Excess-3 converter (truth table → four K-maps → equations)	10 + 10	**§3.7**
2	K-map SOP, minimum literals (4-variable)	6 + 6 + 4	**§4.12**, **§4.20**, **§3.1**
3	JK flip-flop + race-around + master-slave	10 + 10	**§3.13**
4	Shift registers: four types, then the 4-bit bidirectional one	10 + 10 + 4	**§3.4**, **§4.15**
5	Full adder / full subtractor, HA→FA, and the 3:8 decoder version	10 + 10 + 6	**§3.3**, **§4.24**
6	Memory: 16×4 organisation, chip expansion, SRAM vs DRAM	10 + 6 + 6 + 2	**§3.20**, **§3.15**, **§4.23**
7	Converters: R-2R DAC, weighted-resistor DAC, SAR ADC, dual-slope, sample-and-hold	10 + 10 + 10 + 4 + 2	**§3.22**, **§3.19**, **§4.21**, **§3.14**
8	Base conversion + complements (the Q.1 arithmetic)	8–10 every year	**§2.3**–**§2.7**, **§5.1 Q1**

Units II and IV are unavoidable; Unit III and V supply the 10-mark answers; Unit I is free arithmetic; Unit VI is three definitions.

6.2 Q.1 speed sheet — twenty 2-mark answers, one line each

Reproduce the line in under 40 seconds. Anything longer belongs in Q.2–Q.5.

#	Question	One-line answer
1	$(250.5)_{10}$ → binary	250 = 11111010, 0.5 = .1 → **11111010.1**
2	$(101110.0101)_2$ → decimal	46 + 0.25 + 0.0625 = **46.3125**
3	$(25)_{10}$ → binary	16 + 8 + 1 = **11001**
4	$(1101101.111)_2$	**109.875**$_{10}$ = $(155.7)_8$ = $(6D.E)_{16}$
5	$(2AC5.D)_{16}$	**10949.8125**$_{10}$; binary 0010 1010 1100 0101.1101
6	$(330.5)_{10}$ → base 4 / base 8	**$(11022.2)_4$**, **$(512.4)_8$**
7	$(211)_x = (152)_8$	$2x^2 + x + 1 = 106$ → **x = 7**
8	5250 − 321 by 10's complement	5250 + 9679 = 14929, drop carry → **4929**
9	753 − 864 by 10's complement	889, no carry → **−111**
10	11010 − 10000 by 2's / 1's complement	2's: 11010 + 10000 = 101010 → **01010**; 1's: end-around carry → **01010**
11	100 − 110000 (2's comp)	010100, no carry → **−101100 = −44**
12	Universal gate	a gate from which NOT, AND, OR can all be built — NAND, NOR
13	EX-OR from NAND	four NANDs: `w=(ab)'`, `x=(aw)'`, `y=(wb)'`, `F=(xy)'`
14	Tri-state gate	output has 0, 1 **and high impedance Z** → bus sharing
15	AND expression	`Y = A·B`, 1 only when all inputs are 1
16	SOP vs POS	SOP = Σ of minterms (F = 1 rows); POS = Π of maxterms (F = 0 rows)
17	Two combinational circuits	adder, multiplexer (decoder, comparator also fine)
18	MUX function	data selector: 2^n inputs → 1 output via n select lines
19	Flip-flop + two types	bistable 1-bit memory; SR and JK (or D, T)
20	Shift register + application	cascaded FFs sharing one clock; serial↔parallel conversion
21	Sample-and-hold	captures v_in on a capacitor and holds it during conversion
22	Noise margin	`V_NH = V_OH − V_IH = 0.7 V`, `V_NL = V_IL − V_OL = 0.4 V` for TTL
23	Propagation delay	`t_PD = (t_PHL + t_PLH)/2`, TTL 10 ns, ECL 0.5 ns
24	Figure of merit	`t_PD × P_D` in pJ; TTL 100, ECL 25, CMOS 0.075
25	Logic-family characteristics	delay, power, noise margin, fan-out, fan-in, levels, supply, package
26	SRAM vs DRAM	6T latch, no refresh, fast, costly vs 1T1C, refresh, dense, cheap
27	F = (xy+z)(y+xz) as POS	truth table → **ΠM(0,1,2,4)**, Σm(3,5,6,7)
28	Minterm vs maxterm	product = 1 in one row / sum = 0 in one row; $m_i' = M_i$

6.3 The ten answers that must be *exact* (a wrong step loses the whole question)

Item	The exact value to remember
BCD→Excess-3	`W = A + BC + BD`, `X = B'C + B'D + BC'D'`, `Y = CD + C'D'`, `Z = D'`
3-bit squarer	`O5 = AB`, `O4 = AC + AB'`, `O3 = A'BC + AB'C`, `O2 = BC'`, `O1 = 0`, `O0 = C`
Full adder	`S = x⊕y⊕z`, `C = (x⊕y)z + xy` → C = Σm(3,5,6,7)
Full subtractor	`D = x⊕y⊕b`, `B = x'y + x'b + yb` → D = Σm(1,2,4,7), B = Σm(1,2,3,7)
Look-ahead carry	`G_i = A_iB_i`, `P_i = A_i+B_i`, `C1 = G0+P0Cin`, `C2 = G1+P1G0+P1P0Cin`, …
4-bit comparator	`A>B = A3B3' + x3A2B2' + x3x2A1B1' + x3x2x1A0B0'`, `A=B = x3x2x1x0`
Sync counter 0,1,3,4,5,7	`T_A = B`, `T_B = C`, `T_C = B + C'`
Sync 4-bit DOWN (T form)	`T_d = 1`, `T_c = d'`, `T_b = c'd'`, `T_a = b'c'd'`
Gray ↔ binary	`G_i = B_i ⊕ B_{i+1}` ; `B_i = G_i ⊕ B_{i+1}` (cascade from the MSB)
R-2R DAC	`V_out = V_ref · D / 2ⁿ` → with V_ref = 5 V and 4 bits each step is 0.3125 V

6.4 Five diagrams to practise until they are automatic

1. **K-map grids** — 3, 4 and (for the tabulation answer) the labelled 5-variable pair-of-maps. Draw the Gray-order labels without thinking; most lost marks are mis-labelled rows.
2. **JK flip-flop set** — gated JK latch, then the master-slave two-latch version with the feedback, then the race-around waveform (J = K = 1, clock wider than the propagation delay).
3. **4-bit bidirectional shift register** — four FFs, the AND-OR pair per stage, mode M, S1 at the MSB end and S0 at the LSB end (**§4.15**).
4. **Memory chip internal organisation (16×4)** — 4 address lines → 4:16 decoder → 16 × 4 cell array → read/write amplifiers → 4 data lines, with CE and R/W labelled (**§3.20**), and the 8-chip 2048×8 block diagram with the 3:8 decoder for chip select (**§5.4 Q4(a)**).
5. **Converter blocks** — R-2R ladder into an op-amp (**§3.22**), weighted-resistor DAC (**§5.4 Q3(c)**), SAR ADC loop (comparator + SAR register + DAC) (**§3.19**), dual-slope integrator + comparator + counter (**§4.21**), sample-and-hold (**§3.14**).

6.5 Template for a 10-mark design answer

Write in this order and the examiner can always find the marks:

```
1. Problem statement, in your words, one line.
2. Inputs / outputs, with symbols and which one is the MSB.       (1 mark)
3. Truth table — complete, no missing row, don't-cares marked.    (2 marks)
4. One K-map per output, groups circled and labelled.             (3 marks)
5. Simplified Boolean expressions, boxed.                         (1 mark)
6. Logic diagram from those exact expressions.                    (2 marks)
7. Two-line comment: IC family / number of gates / one application.  (1 mark)
```

For a description-type 10-marker (converter, memory, FF): definition → block diagram → working step by step → equations/waveform → advantages → disadvantages → applications. Two of those sections must be something the question did not ask for (the waveform, or the disadvantage) — that is where the last two marks usually sit.

6.6 How marks get lost in this subject

Mistake	Consequence	Fix
Writing maxterms by copying the row bits straight	POS is the complement of what you meant	in a maxterm, complement the literal whose bit is **1**
Don't-care cells left as 0	answer has extra literals and is marked "not minimal"	use them to make bigger groups, then say "d used as 1"
K-map row/column labels in binary order	every group is wrong	always 00 01 11 10
Minterm indices in the wrong bit order	Σm list wrong, whole design wrong	fix the order at the top: "A = MSB = bit 3"
Ripple counter drawn with JK inputs left open	J = K must be 1 for toggle	write J = K = 1 on every stage
S = R = 1 used in an SR table	invalid state	write "not allowed" and move on
Forgetting the end-around carry in (r−1)'s complement	answer off by one	add the carry out back into the LSB
Negative complement result left as the raw sum	sign wrong	no carry out → take the complement again and prefix −
DAC resolution quoted as $V_{ref}/2^n - 1$	off by one step	it is $V_{ref}/2^n$ (full scale is $V_{ref}\cdot(2^n-1)/2^n$)
Copying an expression from the diagram instead of the K-map	diagram and answer disagree	derive the diagram from the boxed equations

6.7 Three-hour plan for this paper

```
0:00 – 0:20   Q.1 (ten 2-mark items). Arithmetic items first, definitions second.
              Never exceed 20 minutes. Each answer: 4 lines, boxed result.
0:20 – 0:35   Q.2 (A) 4-mark minimisation, then (B) 6-mark MUX/decoder.
0:35 – 1:05   Q.2 (C) 10-mark: truth table, maps, equations, diagram.
1:05 – 1:20   Q.3 (A) 4-mark FF conversion or CLA — pick whichever you can write faster.
1:20 – 1:40   Q.3 (B) 6-mark K-map (SOP or POS with don't cares).
1:40 – 2:20   Q.3 (C) 10-mark design. Choose the OR branch you have practised.
2:20 – 2:35   Q.4 (A)+(B) 4 + 6 marks (definitions / family table, then the K-map).
2:35 – 3:00   Q.5: same shape. If time is short, write the 4-mark and 6-mark parts of Q.5 and
              only the block diagram + equations of the 10-marker.
```

Total: 20 + 20 + 20 + 20 + 20 = 100. The internal OR exists to be used — decide on the branch in the first ten seconds and do not switch.

6.8 Two-hour crash list (if the exam is tomorrow)

1. Memorise the BCD→Excess-3 and squarer equations (**§6.3**) — two 10-mark answers, 20 marks.
2. Do four K-maps by hand: **§3.1**, **§3.6**, **§4.12**, **§4.20**. That covers every K-map style asked.
3. Read **§3.13** (JK + race-around + master-slave) and **§4.13** (clocked SR) once, aloud.
4. Learn the two complement rules in **§2.6**/**§2.7** plus the negative case in **§5.1 Q1(e)**.
5. Skim **§6.2** for the Q.1 one-liners — 20 marks in 20 minutes is the cheapest score in the paper.
6. Look at the R-2R and SAR pages (**§3.22**, **§3.19**) and the memory pages (**§3.20**, **§4.23**) for the diagrams only; the text of Unit V answers is description, and description marks come from having a diagram to write around.

6.9 What this book verified, and how

Every number in the answers above was produced or cross-checked by the scripts in `new_exam/` — `eng.py` (the shared Boolean/number-system engine), `verify.py` (the two question papers → `verification.txt`), `verify2.py` (the four assignments → `verification2.txt`) and `verify3.py` (the Winter-2024-only items → `verification3.txt`). Each simplification is not just solved but proved: prime implicants are generated by combining, the cover is checked against the on-set, and an exhaustive search over every subset of prime implicants confirms that nothing shorter exists. Where the exhaustive search disagreed with the greedy cover — the two don't-care questions — the book says so explicitly (**§3.9**, **§5.2 Q2**). Base conversions are round-tripped (convert, then convert back and compare to the original value); counters are simulated state by state, including from unused states; the adder, comparator and decoder designs are evaluated against real integer arithmetic over all input combinations.

Run them again with `python new_exam/verify.py` etc. from the project root; each writes its report file beside the scripts.
